



SENSITIVITY ANALYSIS FOR SCHEDULING PROBLEMS

NICHOLAS G. HALL^{1,*} AND MARC E. POSNER²

¹Max M. Fisher College of Business, The Ohio State University, Columbus, OH 43210, USA

²Industrial Engineering, The Ohio State University, Columbus, OH 43210, USA

ABSTRACT

This paper represents a first attempt at a systematic study of sensitivity analysis for scheduling problems. Because schedules contain both combinatorial and temporal structures, scheduling problems present unique issues for sensitivity analysis. Some of the issues that we discuss have not been considered before. Others, while studied before, have not been explored in the context of scheduling. The applicability of these issues is illustrated using well-known scheduling models. We provide fast methods to determine when a previously optimal schedule remains optimal. Other methods restore an optimal schedule after a parameter change. The value of studying the sensitivity of an optimal sequence instead of the sensitivity of an optimal schedule is demonstrated. We show that, for some problems, sensitivity analysis results depend on the positions of jobs with changed parameters. We identify scheduling problems where performing additional or different computations during optimization facilitates sensitivity analysis. To improve the robustness of an optimal schedule, selection among multiple optimal schedules is considered. We discuss which types of sensitivity analysis questions are intractable because the scheduling problem itself is intractable. We also study how heuristic error bounds vary when the data of a scheduling problem is continuously modified. Although we focus on scheduling problems, several of the issues we discuss and our classification scheme can be extended to other optimization problems.

KEY WORDS: manufacturing and scheduling; sensitivity analysis; optimization; heuristics

OR/MS INDEX 1989 SUBJECT CLASSIFICATION: production/scheduling; sequencing, deterministic; production/scheduling; approximations/heuristic

1. INTRODUCTION

This paper represents a first attempt at a systematic study of sensitivity analysis for scheduling problems. While some of the sensitivity analysis issues we discuss have been considered in other optimization contexts, others have not been considered previously. The study of sensitivity analysis for scheduling problems is motivated in part by the fact that they contain not just a *combinatorial structure* but also a *temporal structure*. Examples of combinatorial structures include which machine is chosen to process a job and the sequence in which the jobs are processed. The temporal structure is when each job is processed, which specifies some parts of a schedule as *local* to others. For example, when the processing time of a job changes, the completion times of only the later jobs on the same machine (assuming no machine idle time) change similarly.

One reason for studying sensitivity analysis for scheduling problems is that the interaction between the combinatorial and temporal structures creates new sensitivity analysis issues. For example, an increase in a processing time of a job may cause a later job to complete after its due

* Correspondence to: Nicholas G. Hall. E-mail: hall.33@osu.edu

date, which necessitates a change in job sequence. Similarly, if we are assigning jobs to two machines, a change in processing time which causes a job completion time to overtake another may necessitate a reassignment of the jobs to the machines. In both cases, the parameter change induces a change in the combinatorial structure through the temporal structure. Another reason for studying sensitivity analysis for scheduling problems is that varying the temporal structure in a continuous way can generate new and interesting sensitivity analysis issues. A third reason is that most scheduling problems are simple enough to permit detailed analysis of the effects of parameter changes. A final reason is that scheduling problems have a wide variety of objectives, constraints and degrees of solvability. These characteristics of scheduling problems generate interesting issues that can be different from those arising in other applications of sensitivity analysis.

Sensitivity analysis is important because real world scheduling problems exist in a changing environment (Anderson et al., 1999). For example, customer demands and specifications, prices of raw materials, and employee and equipment availability, are difficult to estimate and subject to change. As a result, estimates of these values are often used (Hillier and Lieberman, 1995). Sometimes the parameters are even deliberately overstated or understated, to promote the interests of the estimator (Hillier and Lieberman, 1995).

Comprehensive overviews of sensitivity analysis are provided by Gal (1979), Wagelmans (1990), and Greenberg (1998). The latter reference contains an extensive bibliography. For general combinatorial optimization problems, Roberts (1990) and Cozzens and Roberts (1991) use measurement theory to address robustness. Marsten and Morin (1977), Geoffrion and Nauss (1977), and Schrage and Wolsey (1985) study sensitivity analysis issues for branch and bound algorithms. Cutting plane methods are proposed by Klein and Holm (1979), Wolsey (1981), and Cook et al. (1986). Dawande and Hooker (2000) use inference duality to develop sensitivity analysis techniques for mixed integer programming. Ahuja et al. (1993) discuss a variety of sensitivity analysis techniques for network flow problems. Seeger (1996) studies sensitivity analysis in dynamic programming, using subgradients of optimal value functions. Bradley et al. (1977), Wendell (1985, 1992) and Wondolowski (1991) consider sensitivity analysis where multiple parameter changes are allowed. Exact solution methods for such problems are presented by Gal (1979) and Camm and Burwell (1991). Chakravarti and Wagelmans (1998) and Sotskov et al. (1998) consider bounds for an ϵ -optimal solution to remain ϵ -optimal. Van Hoesel and Wagelmans (1999) show that finding the sensitivity analysis range for an individual cost coefficient in an intractable zero-one optimization problem is not possible in polynomial time, unless $P = NP$ (see Garey and Johnson, 1979, for related definitions). They also demonstrate that there does not exist a polynomial time heuristic that comes within a small constant of the actual sensitivity analysis bounds, unless $P = NP$.

There are several works on sensitivity analysis for scheduling problems. Kolen et al. (1994) study the class of scheduling heuristics that is known as list scheduling rules. For several heuristics, they derive bounds on the number of possible assignments of jobs to machines. For a nonlinear optimization model, Sotskov (1991) describes the *stability radius*, that is, the maximum amount of parameter change that maintains optimality. Related works include Kravchenko et al. (1995) and Sotskov et al. (1995). Tovey (1986) considers the scheduling problem of minimizing makespan on a changing number of identical processors. Similar studies of precedence constrained scheduling appear in Graham (1975) and Tovey (1990). In parallel machine environments, Wagelmans (1990) performs parametric analysis of machine speeds and studies the effect of data changes on the quality of heuristic solutions. The sensitivity of an

optimal schedule to linear translations of the cost parameter vector is studied by James and Buchanan (1998) and Mahadev et al. (1998). Kouvelis and Yu (1996) describe branch and bound algorithms for finding the schedule that minimizes the cost of the worst possible solution. They examine single machine and two machine flow shop problems with both interval and discrete processing time data. In a stochastic environment, Daniels and Kouvelis (1995) and Kouvelis et al. (2000) examine similar issues.

In Section 2, we define our notation, describe several classical sensitivity analysis questions, and classify and motivate a variety of issues related to sensitivity analysis in scheduling problems. In Sections 3–6, we provide examples and applications of these issues. Section 3 describes sensitivity analysis results that do not require information about an optimal schedule. Section 4 discusses sensitivity analysis for scheduling problems that are solvable in polynomial time. This section examines several issues, including situations where the position of a job with changing parameters affects sensitivity analysis. We also identify conditions under which standard linear programming sensitivity analysis can be improved, both in establishing optimality ranges and in finding new optimal schedules. Section 5 examines situations where pre-planning for sensitivity analysis makes it worthwhile to perform additional or different computations when finding an optimal schedule. We also study how to find a robust optimal schedule. Section 6 examines the computational complexity of sensitivity analysis for intractable scheduling problems. We study how to derive bounds on the optimal cost after a parameter change, and how to obtain a new heuristic schedule with a guaranteed error bound. Finally, Section 7 contains a conclusion and a discussion of some possible extensions.

2. PRELIMINARIES

In this section, we define our notation, describe several classical sensitivity analysis questions, and introduce the sensitivity analysis issues which we consider.

2.1. Notation

Let $N = \{1, \dots, n\}$ denote the set of jobs to be processed on m machines, $M_1, \dots, M_m$. Let r_j , p_j , w_j , d_j , and $\bar{d}_j$ denote the known release date, processing time, weight, due date, and deadline of job j , respectively, for $j \in N$. We assume throughout that r_j , p_j , w_j , d_j , $\bar{d}_j \geq 0$ and integer for $j \in N$. Let π denote a job *sequence*, or order. A *schedule* σ provides the start and completion time of every job or job piece. For all the scheduling problems we consider, once π and a machine assignment are given, the unique corresponding σ can be determined in $O(n)$ time.

Sensitivity analysis considers the effects of modifying problem parameters. We use the convention that parameters and variables of the modified problem are identified with an added “’”, for example, p_i becomes p_i' . Let $C_j(\sigma)$ denote the completion time of job j in schedule σ before parameter changes occur. The start time of job j in schedule σ is specified by $S_j(\sigma)$. When obvious, $C_j(\sigma)$, $C_j'(\sigma)$, and $S_j(\sigma)$ are replaced by C_j , C_j' , and S_j , respectively. We let σ^* denote an optimal schedule before parameter changes occur and σ' denote an optimal schedule after parameter changes occur. Similarly, π^* and π' represent optimal job sequences before and after there are parameter changes, respectively. Let μ denote the number of simultaneous parameter changes considered. Let $C_j^* = C_j(\sigma^*)$ and $C_j'' = C_j'(\sigma')$. We use similar notation for start times. Let $z(\sigma)$ and $z'(\sigma)$ denote the cost of schedule σ for the original and modified problems, respectively. Also, let $z^* = z(\sigma^*)$ and $z'' = z'(\sigma')$. Let $E_j = \max\{d - C_j, 0\}$ and $T_j = \max\{C_j - \bar{d}, 0\}$

denote the earliness and tardiness of job j , respectively, relative to a common due date, d . Let Δ denote the amount of a parameter change. Also, let $\bar{\Delta}$ and $\underline{\Delta}$ denote the supremum and infimum, respectively, on Δ such that, depending on the context, σ^* , π^* , or z^* remains optimal.

The standard three-field classification scheme for scheduling problems (Graham et al., 1979) is $\psi_1|\psi_2|\psi_3$, where ψ_1 defines the scheduling environment, ψ_2 describes the job characteristics or restrictive requirements, and ψ_3 defines the objective function to be minimized. When $\psi_1 = Pm$, P denotes the identical parallel machine environment. If $m = 1$, then we omit P . For ψ_2 , we use

r_j :	release dates
d_j :	deadlines
$p_j = 1$:	unit processing times
$d \geq \sum p_j$:	unrestrictively large common due date
$pmtn$:	job preemption.

The objectives we consider under ψ_3 are the minimization of

$C_{\max}$:	makespan
$\sum C_j$:	total completion time
$\sum w_j C_j$:	total weighted completion time
$\sum U_j$:	total number of late jobs
$\sum w_j U_j$:	total weighted number of late jobs
$L_{\max}$:	maximum lateness
$\sum (E_j + T_j)$:	total earliness and tardiness.

A scheduling *problem type* is a problem that can be represented by the three-field classification scheme and its extensions. When there is no ambiguity, we frequently omit the word “type.” A scheduling *problem class* is a set of at least two problem types.

2.2. From classical questions to scheduling issues

We describe several classical sensitivity analysis questions that have both theoretical and practical importance. These questions apply to situations where one or more data values change.

- Q1. What are the limits to a parameter change such that the solution remains optimal?
- Q2. Given a specific change of a parameter, what is the new optimal cost?
- Q3. Given a specific change of a parameter, what is a new optimal solution?
- Q4.1–Q4.3. What differences occur in answering Q1–Q3, respectively, when several parameters change simultaneously?

Answering Q1 is useful because it indicates when optimality is no longer being achieved, thus suggesting the need to reoptimize. Typically, when a parameter change exceeds the bounds specified by Q1, a new solution is found by parametric analysis. This approach finds new optimal solutions and values as the parameter changes, until the desired change is reached. Answering Q2 identifies the effect of parameter changes on cost. An answer to Q2 is a solution to the *evaluation version* of a problem, while an answer to Q3 is a solution to the *optimization*

version. A solution to Q3 implies a solution to Q2, however the converse is not true (Papadimitriou and Steiglitz, 1982). Regarding Q3, how a schedule changes may be of interest. Many manufacturing operations need a schedule that does not change too much, that is, one with low “schedule nervousness” (Steele, 1973, Federgruen and Tzur, 1994). For any problem, it is possible to answer Q4.1–Q4.3 by repeating the analysis for each parameter change. However, we are interested in more efficient approaches for scheduling problems. Also, special combinations of parameter changes may be worthwhile to consider.

We now describe a set of interesting and challenging sensitivity analysis issues for scheduling problems. Most of this classification scheme can be extended to more general optimization contexts.

Sensitivity analysis issues for scheduling problems

- I1. Sensitivity analysis results that apply to scheduling problem classes instead of just scheduling problem types.
- I2. Efficient sensitivity analysis approaches when simultaneous parameter changes occur.
- I3. Sensitivity analysis that does not use all the information associated with the optimal schedule.
- I4. Scheduling problems where sensitivity analysis is less useful for σ^* than it is for other output descriptors.
- I5. Local relationships between parts of the optimal schedule that affect sensitivity analysis.
- I6. Optimization approaches motivated by sensitivity analysis.
- I7. Selection of a schedule with minimum sensitivity.
- I8. The computational complexity of answering sensitivity analysis questions for intractable scheduling problems.
- I9. Heuristic sensitivity analysis for intractable scheduling problems.
- I10. Sensitivity analysis of a nonoptimal schedule.

The only issues which have been studied previously for scheduling problems are: issue I2 as it applies to parametric analysis or multiple optimal solutions; issue I7 as it applies to finding a solution that, given a set of possible parameter values, minimizes the worst possible cost; and issue I10.

While most of the issues I1–I10 have been considered in general optimization contexts, I9 has not. Approximate or heuristic sensitivity analysis is applicable to many integer programming problems. Also, while selecting an optimal solution with minimum sensitivity (I7) has been considered previously, it has not been studied by using selection among optimal solutions to provide maximum robustness for one or several parameter changes. Such analysis would be useful for an assignment problem with multiple optimal solutions. The decision about which solution to select could depend on the sensitivity of the cost parameters.

We now provide some motivation for issues I1–I10. Additional motivation is presented as those issues are discussed. Issue I1 investigates sensitivity analysis approaches that can be applied to a class of problems. Such results aid in analyzing each problem in that class.

Issue I2 has been studied for other optimization problems, including network problems (Ahuja et al., 1993). We study situations where multiple parameter changes are independent and others where the changes are dependent. The second case is of particular interest because, in many real world scheduling problems, multiple parameter changes are related. This may be due

to the relationships between the parameters, such as correlation between the value and work required. It may also be due to the modelling process. For example, in a problem with discounted costs, a change in the base cost of a job can induce changes for several time periods.

If issue I3 can be resolved without any knowledge of an optimal solution (*a priori* analysis), then it is possible to estimate the effects of changes in resources or requirements in advance of detailed planning. Geoffrion and Nauss (1977) provide *a priori* results for integer programming. Issue I3 is also important where partial information about σ^* is missing or is unreliable.

Two output descriptors that we consider in issue I4 are π^* and z^* . Sometimes it is more useful to consider sensitivity analysis for the optimal job sequence than for the optimal schedule. Frequently, a production process is controlled by the job sequence and not the actual times provided by a schedule. Thus, the need to reschedule occurs only when π^* changes. Also, even a very small change in a processing time changes an optimal schedule. Larger and more meaningful ranges on parameter changes are obtained when considering π^* . An analogous situation occurs in linear programming. When considering changes to the right-hand side of a tight constraint, it is usual to examine when the basis changes. This is because the optimal solution changes whenever there is a change in the right-hand side (except in the case of degeneracy). Issue I4 also involves situations where it may be more useful to study changes to the optimal cost z^* than to the optimal schedule. We examine situations where there are multiple optimal solutions. Changes are considered for which the original optimal set no longer contains an optimal solution with cost z^* . One application where this issue is important is when we are allocating additional resources to reduce processing times. In this situation, we are interested in the minimum reduction in the processing time of a job that provides a less expensive schedule.

Issue I5 is relevant because of the temporal structure of schedules. For example, this permits analysis of how the positions of jobs with changing parameters affect sensitivity analysis. The analysis may depend on either the original position in σ^* , the new position, or both. The positions may affect the change in optimal cost, the quality of a bound or the computational work. The computational work may be polynomially bounded for some job positions but not for others. The temporal structure also permits isolation of some parts of the schedule from the effects of parameter changes. In practical situations, answering issue I5 can save time and computational effort when the existing information about the original optimal schedule is used. In addition, users of the schedule can continue to implement the parts of the schedule that do not change. Gusfield (1980) examines parts of a solution that remain constant for matroid optimization problems.

Issue I6 is valuable for planning many types of sensitivity analysis in advance, in order to improve the overall efficiency and usefulness of the results. Gusfield (1980) shows how preprocessing can achieve such computational savings for matroid optimization problems. Sometimes it is efficient to store information gathered during the optimization phase, similar to Marsten and Morin (1977). Gusfield (1980) considers how to balance the storage and recomputation of intermediate results. In other cases, the need for sensitivity analysis suggests the use of an entirely different and more complicated optimization procedure. Also, the type of sensitivity analysis performed may influence a choice between solution procedures that are similar in efficiency. Parametric analysis is another example of issue I6. In this case, the goal is to perform a sequence of sensitivity analyses on different optimal schedules in an efficient way. Wagelmans (1990) provides an example of this type of analysis for a two machine makespan problem. A different type of analysis that considers multiple optimal solutions appears in

Gusfield (1980). He studies the tradeoff between the number of optimal solutions found and the cost of finding them.

The sensitivity of a solution to parameter change is important in many applications and is considered in issue I7. For example, Kouvelis and Yu (1996) formulate and solve models for minimizing the worst possible cost, given a set of parameter values. However, we consider a new variation of this issue, where a problem has multiple optimal solutions. In this case, it may be useful to choose one which is most robust to a particular parameter change. Negative results that show the intractability of making such a choice are also valuable. As described later, the analysis of these topics in our work depends on the temporal structure of scheduling problems.

For issue I8, understanding the computational complexity of sensitivity analysis directs the design of appropriate types of sensitivity analysis methods. Similar work exists for zero–one optimization problems (Van Hoesel and Wagelmans, 1999). The large number of computational complexity results that are available for scheduling problems can be used to imply related results for sensitivity analysis.

Such intractability results for sensitivity analysis problems motivate the use of heuristic sensitivity analysis approaches. Regarding issue I9, one useful type of analysis is to derive heuristic bounds on the quality of solutions obtained by simple sensitivity analysis procedures. The temporal structure of an optimal schedule makes it possible to observe and analyze the effect of continuous parameter changes in a way that may not be possible for a generic optimization problem. As a result, we are able to identify a range of data over which a worst case performance bound holds. It is also interesting to show when the limits which answer Q1 are only sufficient and when they are exact.

An example of Issue I10 is studying sensitivity analysis for a heuristic schedule (Kolen et al., 1994; Chakravarti and Wagelmans, 1998; Sotskov et al., 1998). Since issue I10 has been studied by other authors, we do not consider it in the remainder of the paper.

2.3. Overview of the results

Table I summarizes, for our results, the connections between the classical sensitivity analysis questions and issues I1–I10. Because of space limitations, we address only a subset of example applications. However, we provide at least one application for each issue (except for I10, that

Table I. Results connecting questions and issues

	Q1	Q2	Q3	Q4.1	Q4.2	Q4.3
I1				Thm 4.1		
I2				Thm 4.3, 4.9	Thm 5.1	Thm 4.7, 4.8, 5.1
I3 R1–R10				Thm 3.1		
I4 R10, Thm 4.5, 5.2, 5.3, 5.4			Thm 4.6	Thm 4.3		
I5		Thm 4.2, 6.2		Thm 3.1	Thm 4.4	Thm 4.4
I6 Thm 5.2					Thm 5.1	Thm 5.1
I7 Thm 5.3, 5.4						
I8		Thm 6.1	Thm 6.1			
I9 Thm 6.3, 6.4		Thm 6.2, 6.5, 6.6	Thm 6.6			
I10 Kolen et al. (1994), etc.						

has several applications in the literature). Where multiple results appear within the same row, they illustrate different aspects of an issue.

We now discuss some of these different aspects. Regarding issue I2, Theorem 4.3 considers two dependent parameter changes for a list scheduling problem. In contrast, Theorem 4.7 considers multiple independent changes for a scheduling problem solved by linear programming. When the use of a concise formulation means that one parameter change implies others, Theorem 4.8 considers schedule reoptimization, and Theorem 4.9 considers optimality ranges for changing parameters. Also, Theorem 5.1 shows how to expedite sensitivity analysis by storing additional information when parameters change simultaneously. For issue I3, results R1–R10 use no information about the solution, while Theorem 3.1 only uses information about the last job on each machine. For issue I4, result R10 considers a situation with limited information about σ^* . In Theorem 4.3, only information about adjacent jobs is needed. Theorem 4.5 demonstrates how studying an optimal sequence instead of an optimal schedule can provide necessary and sufficient conditions for continued optimality. Theorem 4.6 shows how efficient reoptimization is possible using the optimal sequence. Theorem 5.2 demonstrates the usefulness of studying the optimal cost instead of an optimal schedule. Finally, Theorems 5.3 and 5.4 provide positive and negative results, respectively, about finding the most robust sequence. For issue I5, Theorems 3.1 and 4.4 consider a schedule with multiple parameter changes for different classical questions. In contrast, Theorem 4.2 considers the effect of job position on the cost of a single parameter change. The effect of job position on heuristic analysis is considered in Theorem 6.2. For issue I6, Theorem 5.1 shows how to expedite sensitivity analysis by storing additional information, whereas Theorem 5.2 suggests a new optimization procedure that facilitates sensitivity analysis. For issue I7, Theorem 5.3 shows how to identify a robust schedule from a set of optimal schedules, whereas Theorem 5.4 identifies situations where this is not possible. For issue I9, Theorem 6.2 finds a heuristic bound on the cost of a new optimal schedule after a given parameter change. Theorems 6.3 and 6.4 find two different exact limits on parameter change for continued optimality in heuristic sensitivity analysis. Finally, Theorem 6.5 provides heuristic bounds on the cost of the current optimal schedule, while Theorem 6.6 provides heuristic bounds on the cost of a modified schedule.

3. A PRIORI RESULTS

Motivated by issue I3, we examine what can be said about Q1–Q3 *before* a specific optimal schedule is found. Geoffrion and Nauss (1977) provide some answers to this question. They discuss sensitivity analysis results based on elementary observations for general integer programs. The basic results they present, stated using scheduling terminology, are:

- GN1: If the set of feasible schedules is reduced and σ^* is still feasible, then σ^* is still optimal.
- GN2: In a minimization problem, if job k completes at its lower bound in σ^* and the cost of k increases, then σ^* is still optimal.
- GN3: In a minimization problem, if job k completes at its upper bound in σ^* and the cost of k decreases, then σ^* is still optimal.

We adapt these results to scheduling problems. For most scheduling problems with regular measures (see Rinnooy Kan, 1976, for a definition), the following parameter changes do not alter an optimal schedule.

From GN1:

- R1: An increase in a release date, but not later than the time when the job starts processing.
- R2: A decrease in a deadline, but not earlier than the time when the job completes processing.
- R3: The addition of a precedence constraint requiring job i to be processed before job k when $C_i^* \leq S_k^*$.

From GN2:

- R4: An increase in the weight of a job that is processed first.
- R5: An increase in the weight of a job that starts processing at its release date.

From GN3:

- R6: A decrease in the weight of a job that completes processing at its deadline.
- R7: A decrease in the weight of a job that is processed last in a single machine scheduling problem.

It is possible to derive additional results, similar to those above, that do not follow from GN1–GN3. Result R8 describes a sufficient condition under which an increase in the set of feasible schedules does not change the optimal schedule in nonpreemptive scheduling problems that have regular measures and do not have precedence constraints.

- R8: An increase in $\bar{d}_k$ when there is consecutive idle time of at least $\max_{j \in N} \{p_j | C_j^* > C_k^*\}$ in the interval $[C_k^*, \bar{d}_k]$.

A bottleneck cost problem is one where the objective is to minimize the largest single cost incurred in the schedule. An example is minimizing the maximum lateness.

- R9: In bottleneck objective problems, an increase in processing time that does not create a new bottleneck or increase the current bottleneck.

Some extensions of these results are possible. An example is a multiple machine version of R7. The following result addresses issues I3 and I5 by answering Q4.1. This result is an example where sensitivity analysis is polynomially bounded for specific job positions and type of analysis. Contrast this result with the intractability of Q1 for changes in job weights, as proved by Van Hoesel and Wagelmans (1999).

Theorem 3.1. For identical parallel machine scheduling problems with $\sum w_j C_j$ objective, if the weight of the last job on each machine is reduced by the same constant Δ , then σ^* remains optimal.

Proof. Reindex the jobs so that i is the last job to complete on machine M_i in σ^* for $i = 1, \dots, m$. Suppose we reduce the weight of jobs $1, \dots, m$ by Δ and σ^* is no longer optimal. Let σ' be optimal, and let h_i be the last job to finish on M_i in σ' for $i = 1, \dots, m$. Now, $\sum_{i=1}^m C_{h_i}'' = \sum_{j=1}^n p_j$. From the optimality of σ' , $|C_{h_i}'' - C_{h_q}''| \leq \min\{p_{h_i}, p_{h_q}\}$ for $1 \leq i \neq q \leq m$. Thus, $\sum_{i=1}^m C_i'' \leq \sum_{i=1}^m C_{h_i}''$. This implies that $\sum_{i=1}^m \Delta C_i'' \leq \Delta \sum_{j=1}^n p_j$. Since σ^* is no longer optimal,

$$z'(\sigma') < z'(\sigma^*) = z(\sigma^*) - \Delta \sum_{j=1}^n p_j.$$

If we increase the weight of jobs $1, \dots, m$ by Δ , then

$$z(\sigma') = z'(\sigma') + \Delta \sum_{i=1}^m C_i'' \leq z'(\sigma') + \Delta \sum_{j=1}^n p_j < z(\sigma^*).$$

This contradicts the optimality of σ^* . ■

The following result describes a parameter change that does not alter π^* (issue I4).

R10: An increase of Δ in p_k , when there is idle time throughout the interval $[C_k^*, C_k^* + \Delta]$.

4. PROBLEMS THAT ARE SOLVABLE IN POLYNOMIAL TIME

We first consider the class of scheduling problems that are polynomially solvable by a list scheduling algorithm. We address issue I1 for all scheduling problem types in this class. Then, we refine our analysis for a scheduling problem that has a nonregular performance measure, addressing issue I5. Next, an example of issue I4 is discussed. This example also addresses issue I2 when there are dependent parameter changes. The same scheduling problem is used to address issue I5. Next, we consider polynomially solvable scheduling problems that require an algorithm more complicated than list scheduling, and present examples for issue I4. Finally, we examine scheduling problems that can be solved in polynomial time by variants of linear programming. For this class of scheduling problems, we consider issue I2 for questions Q4.1 and Q4.3. We also provide an improvement on the optimality range found by classical sensitivity analysis and its general purpose extensions.

4.1. Problems admitting optimal list schedules

The simplest scheduling problem class that we consider are those that are optimally solvable by a *list scheduling rule*. Problems in this class are called list scheduling problems. For these problems, a priority list is determined from pairwise comparisons of the characteristics of each individual job. Then, jobs are selected for processing in list sequence. The temporal structure of the schedule creates adjacency between pairs of jobs, which is useful in performing sensitivity analysis, especially in the verification of continued optimality. Some scheduling problem types that are solvable by a list scheduling rule include $1||\sum w_j C_j$ (Smith, 1956), $P||\sum C_j$ (Conway et al., 1967), $1||L_{\max}$ (Jackson, 1955) and $1|d \geq \sum p_j|\sum (E_j + T_j)$ (Kanet, 1981).

To analyze the time complexity required to answer sensitivity analysis questions for list scheduling problems, we introduce some notation. Let $\mathcal{S}$ be the computational time required to compare two jobs in a list. In the scheduling problems we consider, $O(\mathcal{S})$ is a constant. Given a list, let $\mathcal{T}$ be the time required to determine both a schedule, which includes all job start or completion times, and its cost. Throughout this work, $\mathcal{T} = O(n)$. The first result addresses issue I1 by establishing the time complexity of answering Q4.1–Q4.3 for all list scheduling problems.

Theorem 4.1. For list scheduling problems, if a set of parameter changes does not alter the list sequence, then the continued optimality of σ^ can be verified in $O(\mu\mathcal{S})$ time. Otherwise, a new optimal schedule and its cost can be determined in $O(\mu\mathcal{S} \log n + \mathcal{T})$ time.*

Proof. We can determine whether the position of a job changes by making comparisons between adjacent jobs in the list in $O(S)$ time. Consequently, the optimality of σ^* can be verified in $O(\mu S)$ time. When a schedule is no longer optimal, it is necessary to resequence some jobs. The new optimal position for each such job can be found by pairwise comparisons with other jobs in $O(S \log n)$ time. Since there are at most μ such jobs, this requires $O(\mu S \log n)$ time. A new optimal schedule can then be found in $O(T)$ time. ■

The insight from Theorem 4.1 is that only *adjacent* comparisons are needed to verify optimality. It would be interesting to find other optimization problems where this is the case. Although Theorem 4.1 answers Q4.1–Q4.3 for list scheduling problems, there are other issues to address. One issue is when does sensitivity analysis depend on the position of the job with changing parameters. For $1|d \geq \sum p_j| \sum (E_j + T_j)$, we show how to answer Q2 in $O(\log n)$ time. This is less than the $O(S + T) = O(n)$ time given by Theorem 4.1. Algorithm KANET (Kanet, 1981), finds an optimal schedule for this scheduling problem.

Algorithm KANET

1. Reindex the jobs such that $p_1 \geq \dots \geq p_n$.
2. Job $2i - 1$ is placed in position i , for $i = 1, \dots, \lceil n/2 \rceil$.
Job $2i$ is placed in position $n - i + 1$, for $i = 1, \dots, \lfloor n/2 \rfloor$.
3. Schedule job $\lceil n/2 \rceil$ to complete at d . Schedule all other jobs without inserted idle time.

We first describe a procedure that finds a new optimal job sequence when there is a single parameter change of the form $p'_k = p_k + \Delta$, where $\Delta > 0$. The case, where $\Delta < 0$ is similar.

Algorithm ETCHG

1. Find the position of p'_k in the nonincreasing processing time sequence. Let i denote this position, where $i < k$.
2. Jobs $1, \dots, i - 1$, and jobs $k + 1, \dots, n$ retain their positions in σ' .
3. Job k occupies in σ' the position that job i occupies in σ^* .
4. Jobs from $i, \dots, k - 1$ that preceded d in σ^* follow d in σ' , and vice versa. These jobs are sequenced by increasing indices before d and decreasing indices after d .

We need the following preliminary result.

Lemma 4.1. For some $k \in N$, let $p'_k = p_k + \Delta$ be a parameter change for $1|d \geq \sum p_j| \sum (E_j + T_j)$, where $\Delta > 0$. Then, Algorithm ETCHG finds a new optimal job sequence.

Proof. By construction. ■

Let C_{-1}^* denote the start time of job 1. The next result addresses issue I5.

Theorem 4.2. For some $k \in N$, let $p'_k = p_k + \Delta$ be a parameter change for $1|d \geq \sum p_j| \sum (E_j + T_j)$, where $\Delta > 0$. Also, let i be the new position of job k in the nonincreasing processing time sequence.

The cost of a new optimal schedule is

$$z'' = z^* + \lfloor i/2 \rfloor \Delta - \lceil (k-i)/2 \rceil p_k + \begin{cases} C_{k-2}^* - C_{i-2}^*, & \text{if } k \text{ and } i \text{ are both odd} \\ C_{k-1}^* - C_{i-2}^*, & \text{if } k \text{ is even and } i \text{ is odd} \\ C_{k-1}^* - C_{i-1}^*, & \text{if } k \text{ and } i \text{ are both even} \\ C_k^* - C_{i-1}^*, & \text{if } k \text{ is odd and } i \text{ is even.} \end{cases}$$

Further, this new optimal cost can be found in $O(\log n)$ time.

Proof. From Lemma 4.1, ETCHG finds a new optimal schedule. In each of the four cases in the statement of the theorem, the costs of jobs $k+1, \dots, n$ do not change. As part of this analysis, observe that $p_k = C_k^* - C_{k-2}^*$ when $k \geq 1$ is odd, and $p_k = C_k^* - C_{k+2}^*$ when $k \geq 2$ is even.

Case 1. k and i are odd, that is $C_i^*, C_k^* < d$.

The total change in cost for jobs $1, 3, \dots, i-2$ is

$$\left\lfloor \frac{i}{2} \right\rfloor [(C_{i+1}^* - C_{k+1}^*) - (C_{k-2}^* - C_{i-2}^*) + \Delta].$$

The total change in cost for jobs $2, 4, \dots, i-1$ is

$$\left\lfloor \frac{i}{2} \right\rfloor [(C_{k-2}^* - C_{i-2}^*) - (C_{i+1}^* - C_{k+1}^*)].$$

The total change in cost for jobs $i, i+2, \dots, k-2$ is

$$\frac{(k-i)}{2} [(C_{k+1}^* - d) - (d - C_{k-2}^*)] + (C_{k-2}^* - C_{i-2}^*).$$

The total change in cost for jobs $i+1, i+3, \dots, k-1$ is

$$\frac{(k-i)}{2} [(d - C_k^*) - (C_{k+1}^* - d)] - (C_{i+1}^* - C_{k+1}^*).$$

The total change in cost for job k is $C_{i+1}^* - C_{k+1}^*$. Adding the above terms establishes Case 1 of the theorem. The proofs of the other three cases are similar.

Although ETCHG constructs a new optimal schedule, the cost of that schedule can be computed from z^* without implementing ETCHG. Finding i , the new position of job k , requires $O(\log n)$ time. Given i , the optimal cost can be computed in constant time. Thus, the overall time requirement is $O(\log n)$. ■

The temporal structure of a schedule, which determines the cost change resulting from a pairwise interchange of jobs, makes Theorem 4.2 possible. It would be interesting to study other optimization problems where the cost of a new optimal solution is found without finding the new solution itself.

While Theorem 4.1 gives general guidelines, additional analysis may be needed to provide efficient results for specific scheduling problems. Such an example is $1||\sum w_j C_j$. Suppose we simultaneously change parameters p_k and w_k for a given job $k \in N$. In a real world example, the value of a job may be based on the raw material cost plus a term for the added labor. Thus,

$$w_k = \tau p_k + K, \tag{1}$$

where K is the raw material cost and τ is the added value per unit of processing time. If p_k changes, then we expect w_k to change also. When the value of the weight is given by (1), the new value of w_k is $w'_k = w_k + \tau\Delta$ if $p'_k = p_k + \Delta$.

Assume that jobs are indexed such that $w_1/p_1 \geq \dots \geq w_n/p_n$. We now provide an example of issues I2 and I4 that answers Q4.1.

Theorem 4.3. For some $k \in N$, let $p'_k = p_k + \Delta$ and $w'_k = w_k + \tau\Delta$ be parameter changes for $1 \parallel \sum w_j C_j$. Then, π^* remains optimal if and only if Δ is within the following ranges.

i. If $\tau < w_{k+1}/p_{k+1}$, then

$$(w_k p_{k-1} - w_{k-1} p_k) / (w_{k-1} - \tau p_{k-1}) \leq \Delta \leq (w_k p_{k+1} - w_{k+1} p_k) / (w_{k+1} - \tau p_{k+1}).$$

ii. If $w_{k+1}/p_{k+1} \leq \tau \leq w_{k-1}/p_{k-1}$, then

$$\max\{(w_{k+1} p_k - w_k p_{k+1}) / (\tau p_{k+1} - w_{k+1}), (w_k p_{k-1} - w_{k-1} p_k) / (w_{k-1} - \tau p_{k-1})\} \leq \Delta < \infty.$$

iii. If $\tau > w_{k-1}/p_{k-1}$, then

$$(w_{k+1} p_k - w_k p_{k+1}) / (\tau p_{k+1} - w_{k+1}) \leq \Delta \leq (w_{k-1} p_k - w_k p_{k-1}) / (\tau p_{k-1} - w_{k-1}).$$

Furthermore, the above conditions can be checked in constant time.

Proof. The stated inequalities follow from the conditions that keep the list order unchanged. ■

Theorem 4.3 provides bounds on modifications to the parameters of job k such that the optimal sequence does not change. When the sequence does change, it is interesting to consider how much the schedule is disrupted, and how much this disruption affects the computational work required (issue I5). The next result examines the extent of schedule disruption that results from a parameter change and provides specific answers to Q4.2 and Q4.3.

Theorem 4.4. For some $k \in N$, let $p'_k = p_k + \Delta$ and $w'_k = w_k + \tau\Delta$ be parameter changes for $1 \parallel \sum w_j C_j$. If $w_{j-1}/p_{j-1} \geq w'_k/p'_k \geq w_j/p_j$, then the times at which the jobs $1, \dots, \min\{k-1, j-1\}$ are processed do not change. Further, jobs $\max\{k+1, j\}, \dots, n$ are processed in the same sequence. Also, if $j < k$, then jobs $j, \dots, k-1$ are processed in the same sequence, and if $j > k$, then jobs $k+1, \dots, j-1$ are processed in the same sequence. The time required to compute a new optimal schedule and its cost is proportional to $|k-j|$.

Proof. The new job position and sequence follow from the fact that a WSPT sequence is optimal.

The cost of a new optimal schedule is

$$z^* + \begin{cases} p_k \sum_{i=j}^{k-1} w_i - w_k \sum_{i=j}^{k-1} p_i, & j < k \\ -p_k \sum_{i=k+1}^{j-1} w_i + w_k \sum_{i=k+1}^{j-1} p_i, & j > k. \end{cases}$$

■

Theorem 4.4 is an example of position dependent sensitivity analysis, which is possible because the temporal structure determines the cost change resulting from a pairwise interchange of jobs. The time required to compute the new optimal schedule and cost depends on the change in position of job k between the original and the modified list sequence. As with Theorem 4.2, Theorem 4.4 motivates the search for other optimization problems for which the cost of a new optimal solution can be found without finding the solution itself.

4.2. Other problems that are solvable in polynomial time

The analysis in Theorem 4.1 demonstrates that sensitivity analysis for list scheduling problems can be performed quickly, and sometimes without using all of the information about σ^* . However, for scheduling problems that require more complicated but still polynomial time procedures, the full details of σ^* may be essential for efficient sensitivity analysis. This provides limits on the analysis of issue I3. A scheduling problem of this type is $1|r_j, pmtn|\sum C_j$. The shortest remaining processing time (SRPT) rule proposed by Miller and Schrage (1966) finds an optimal schedule for this problem in $O(n \log n)$ time. For this problem, we first examine when π^* changes, issue I4. By performing sensitivity analysis for π^* , we answer Q1 in Theorem 4.5.

The following result characterizes an optimal schedule for problem $1|r_j, pmtn|\sum C_j$.

Lemma 4.2. (Schrage, 1968) An optimal schedule for $1|r_j, pmtn|\sum C_j$ can be found by processing the available job with the shortest remaining processing time at each point in time. Further, this schedule contains no more than $n - 1$ preemptions.

The next result identifies a useful relationship between π^* and σ^* . While π^* does not completely characterize a schedule in preemptive problems, it does so for $1|r_j, pmtn|\sum C_j$ when σ^* is found using the SRPT rule.

Lemma 4.3. An optimal schedule for $1|r_j, pmtn|\sum C_j$ is uniquely characterized by a job sequence.

Proof. From the SRPT rule, a job can be preempted by job j only at time r_j . Hence, each period of processing that does not complete a job ends at the release date of the subsequent job in the sequence. Thus, the sequence of jobs processed completely characterizes a schedule. ■

When the release date for a job changes, the next preliminary result establishes time intervals within which the optimal schedule does not change.

Lemma 4.4. For some $k \in N$, let $r'_k = r_k + \Delta$ be a parameter change for $1|r_j, pmtn|\sum C_j$. Suppose that $\pi' = \pi^$. If job k preempts job i in σ^* , then σ' and σ^* are identical outside the time interval $[\min\{r_k, r'_k\}, C_i^*]$.*

Proof. Because the selections by the SRPT rule are unchanged in the interval $[0, r_k]$, σ' and σ^* are identical before the release date of job k . Since, $\pi' = \pi^*$ by assumption, only jobs that are processed in the interval $[\min\{r_k, r'_k\}, C_i^*]$ for π^* are processed in this interval for π' . Further, the SRPT rule ensures that if job j is processed in $[\min\{r_k, r'_k\}, C_i^*]$, then j starts and completes in that interval. Thus, after C_i^* , σ' , and σ^* are identical. ■

Lemma 4.4 is an example of a position dependent result for a scheduling problem that both is solvable in polynomial time and does not have a known optimal list scheduling rule. Lemma 4.4 illustrates issue I5. Unlike Theorem 4.4, this result depends only on the location of the job in the original optimal schedule and not on its new location.

We assume that the jobs are indexed such that $r_1 \leq \dots \leq r_n$. Moreover, if $r_j = r_{j+1}$, then $p_j \leq p_{j+1}$. To break ties when using the SRPT rule, we assume that the job with the smallest index is selected for processing. Define σ^* and σ' to be optimal SRPT schedules, where $r'_k = r_k + \Delta$. Also, job i is the last job in process before S_k^* . Let

- $\hat{S}_j(t)$ = start time of the last period of processing of job j before time t in σ
- $R_j(t)$ = remaining processing time of job j at time t in σ^*
- $\alpha_j(t)$ = size of the first piece of job j that starts no earlier than time t in σ^* , if there are multiple pieces
- $I(t_0, t_1)$ = total idle time in the interval $[t_0, t_1]$ in σ^*
- $\mathcal{E} = \{j \mid \text{job } j \text{ starts earlier in } \sigma' \text{ than in } \sigma^* \text{ for some } \Delta < 0\}$
- e = first job to begin processing in σ^* after all jobs in $\mathcal{E}$, and before C_i^* when $\mathcal{E} \neq \emptyset$
- $\mathcal{L} = \{j \mid \text{job } j \text{ starts later in } \sigma' \text{ than in } \sigma^* \text{ for some } \Delta > 0\}$.

We first show how to calculate $\mathcal{E}$ and $\mathcal{L}$, given $\pi' = \pi^*$ and a parameter change of the form $r'_k = r_k + \Delta$, for some $k \in N$, for $1|r_j, pmtn|\sum C_j$.

Procedure FIND $\mathcal{E}$: for $\Delta < 0$

0. Let job i be the last job in process before job k starts. Set $\mathcal{E} = \emptyset$.
1. If $C_i^* = S_k^*$, then terminate.
2. Set $\mathcal{E} = \{k\}$. Let $j = k$.
3. Let q be the job such that $C_j^* = S_q^*$. If $r_q = S_q^*$ or if q does not exist, then terminate.
4. Set $\mathcal{E} = \mathcal{E} \cup \{q\}$, and $j = q$. Go to Step 3.

Procedure FIND $\mathcal{L}$: for $\Delta > 0$

1. Set $\mathcal{L} = \{k\}$. Let $j = k$.
2. Let $q = \min\{l \in N \mid C_j^* \leq S_l^*\}$ and no processing occurs in the interval $[C_j^*, S_l^*]$. If q does not exist, then terminate.
3. Set $\mathcal{L} = \mathcal{L} \cup \{q\}$, and $j = q$. Go to Step 2.

Next, we present two lemmas that identify necessary and sufficient conditions for π^* to remain optimal after a release date change.

Lemma 4.5. For some $k \in N$, let $r'_k = r_k + \Delta$ be a parameter change for $1|r_j, pmtn|\sum C_j$. Suppose $\Delta < 0$ and suppose i is the last job that is processed before r_k in σ^ . Then, π^* is an optimal sequence for the modified scheduling problem if and only if*

- a. $\Delta \geq \min\{t < S_i^* \mid I(t, S_i^*) = 0\} - r_k$, when $S_k^* \geq C_i^*$
- b. $\Delta \geq \max\{t \mid R_j(t) \geq p_k \text{ for some } j \text{ where } C_j^* \leq S_k^*\} - r_k$, when $S_k^* \geq C_i^*$
- c. $\Delta > \max_{q \in \mathcal{E}} \max_{u \in N} \{p_u - R_q(r_u) \mid S_q^* < S_u^* < C_q^*, \text{ no } j \in \mathcal{E} \text{ exists such that } C_j^* \leq S_q^* \text{ and } S_j^* - r_j < R_q(r_u) - p_u\}$
- d. $\Delta \geq \max_{q \in \mathcal{E} \cup \{e\} \setminus \{k\}} \{r_q - S_q^* \mid S_k^* < C_i^* \text{ or there exists a } j \text{ where } r_j < r_q \text{ and } C_j^* > C_q^*\}$
- e. $\Delta > \hat{S}_i^*(r_k) - r_k$, when $S_k^* < C_i^*$
- f. $\Delta \geq \max_{q \in N} \{R_i(r_q) - p_q \mid S_i^* < r_q < C_i^* \leq S_q^*\}$, when $S_k^* < C_i^*$.

Proof. The specifications of strict or weak inequalities in parts b, c, and f are due to the tie breaking rule that we choose.

($\Leftarrow$). For each part, we describe why π^* is no longer optimal when the stated condition is violated.

- a. A piece of job k is processed before S_i^* .
- b. Job k preempts job j , where we allow $j = i$.
- c. Some job u , where $S_q^* < S_u^* < C_q^*$, no longer preempts job q . However, if there exists some $j \in \mathcal{E}$ such that $C_j^* \leq S_q^*$ and $S_j^* - r_j < R_q(r_u) - p_u$, then job j blocks job q from moving earlier enough to change π^* .
- d. Some job $q \in \mathcal{E}$ starts earlier in σ' by an amount that is less than Δ . This creates a gap of size $r_q - S_q^* - \Delta > 0$ before q in the schedule. Some job j such that $R_j(r_q) > 0$ can be inserted in this gap. Note that job q does not preempt any jobs in σ^* , because $q \in \mathcal{E} \cup \{e\}$. Thus, job j is not processed immediately before job q in σ^* . Hence, π^* changes. If $C_i^* > S_k^*$, then job i can be processed in the gap before job q .
- e. The piece of i that starts at $\hat{S}_i(r_k)$ is replaced by job k .
- f. Since the remaining processing time of job i at S_k^* is increased, i is no longer selected over job q which is released after S_i^* and which starts after C_i^* .

($\Rightarrow$). We establish that if the conditions in parts a–f are satisfied, then π^* is optimal for the modified scheduling problem. The jobs are partitioned into three subsets: those that start before S_i^* , job i , and those that start after S_i^* . Note that the sequence after job e does not change, because e starts at the same time in σ^* and σ' . Also from Lemma 4.4, the jobs processed before r_k' do not change when $S_k^* < C_i^*$.

For the jobs that start before S_i^* , there are three cases. First, if $S_k^* \geq C_i^*$, then the condition in part a prevents k from starting during idle time before S_i^* . Second, if $S_k^* \geq C_i^*$, then the condition in part b prevents job k from preempting i or a job that is processed before i . Finally, if $S_k^* < C_i^*$, then the condition in part e prevents k from delaying the piece of i that starts at $\hat{S}_i$.

For job i , there are three cases. First, if $S_k^* \geq C_i^*$, then the condition in part b prevents k from delaying the processing of i . Alternatively, if $S_k^* < C_i^*$, then k can start earlier. The condition in part e prevents k from delaying the last piece of i before S_k^* . The last possibility is that some job q displaces i because the remaining processing time of i has increased. The condition in part f prevents this situation.

For the jobs that start after S_i^* , there are two cases. First, the condition in part c ensures that a job u which preempts a job $q \in \mathcal{E}$ in σ^* continues to do so. Second, some job $q \in \mathcal{E} \cup \{e\} \setminus \{k\}$ may not start early enough to prevent idle time in the schedule. The condition in part d prevents any job from being processed during this idle time period. ■

Lemma 4.6. For some $k \in N$, let $r'_k = r_k + \Delta$ be a parameter change for $1|r_j, \text{pmtn}|\sum C_j$. Suppose $\Delta > 0$ and suppose i is the last job that is processed before r_k in σ^* . Then, π^* is an optimal sequence for the modified scheduling problem if and only if

- a. $\Delta \leq \max\{S_k^*, \min_{j \in N} \{r_j | R_j(S_k^*) > 0\}\} - r_k$, when $S_k^* \geq C_i^*$
- b. $\Delta \leq S_k^* - r_k + \min_{q \in \mathcal{L}, u \in N} \{I(r_k, r_q) + \max\{r_u - C_q^*, 0\} + p_u - R_q(r_u) | C_q^* < S_u^*, p_u \leq p_q\}$
- c. $\Delta < S_k^* - r_k + \min_{q \in \mathcal{L}} \{I(r_k, r_q) + \alpha_q(r_q)\}$
- d. $\Delta < \alpha_i(C_k^*)$, when $S_k^* < C_i^*$
- e. $\Delta < \min_{q \in \mathcal{L}} \{R_i(r_q) - p_q | S_k^* \leq S_q^* < C_i^*\}$, when $S_k^* < C_i^*$.

Proof. Similar to that of Lemma 4.5. ■

We provide sufficient conditions for π^* to remain optimal after a single release date change. That is, we describe conditions under which the SRPT rule selects the same job sequence for the original data and the modified data. In part i of Theorem 4.5, job k does not preempt job i in σ^* . In part ii, job k preempts job i at r_k in σ^* . From the SRPT rule, job k does not preempt job i at any time other than r_k . Therefore, these two cases are exhaustive.

Theorem 4.5. For some $k \in N$, let $r'_k = r_k + \Delta$ be a parameter change for $1|r_j, pmtn| \sum C_j$. Suppose that i is the last job that is processed before r_k in σ^* . Then, π^* is optimal for the modified scheduling problem if and only if one of the following conditions holds for an arbitrarily small $\epsilon > 0$.

i. $S_k^* \geq C_i^*$ and

$$\begin{aligned} & \max \left\{ \min\{t < S_i^* | I(t, S_i^*) = 0\} - r_k, \max\{t | R_j(t) \geq p_k \text{ for some } j \text{ where } C_j^* \leq S_k^*\} - r_k, \right. \\ & \max_{q \in \mathcal{E}} \max_{u \in N} \{p_u - R_q(r_u) | S_q^* < S_u^* < C_q^*, \text{ no } j \in \mathcal{E} \text{ exists such that } C_j^* \leq S_q^* \text{ and} \\ & \left. S_j^* - r_j < R_q(r_u) - p_u\} + \epsilon, \max_{q \in \mathcal{E} \cup \{e\} \setminus \{k\}} \{r_q - S_q^* | \text{there exists a } j \text{ where } r_j < r_q \text{ and } C_j^* > C_q^*\} \right\} \\ & \leq \Delta \leq \min \left\{ \max\{S_k^*, \min_{j \in N} \{r_j | R_j(S_k^*) > 0\}\} - r_k, \right. \end{aligned}$$

$$\begin{aligned} & S_k^* - r_k + \min_{q \in \mathcal{L}, u \in N} \{I(r_k, r_q) + \max\{r_u - C_q^*, 0\} + p_u - R_q(r_u) | C_q^* \leq S_u^*, p_u < p_q\}, \\ & \left. S_k^* - r_k + \min_{q \in \mathcal{L}} \{I(r_k, r_q) + \alpha_q(r_q)\} - \epsilon \right\}. \end{aligned}$$

ii. $S_k^* < C_i^*$ and

$$\begin{aligned} & \max \left\{ \max_{q \in \mathcal{E}} \max_{u \in N} \{p_u - R_q(r_u) | S_q^* < S_u^* < C_q^*, \text{ no } j \in \mathcal{E} \text{ exists such that } C_j^* \leq S_q^* \text{ and} \right. \\ & \quad S_j^* - r_j < R_q(r_u) - p_u\} + \epsilon, \max_{q \in \mathcal{E} \cup \{e\} \setminus \{k\}} \{r_q - S_q^*\}, \hat{S}_i^*(r_k) - r_k + \epsilon, \\ & \left. \max_{q \in N} \{R_i(r_q) - p_q | S_i^* < r_q < C_i^* \leq S_q^*\} \right\} \leq \Delta \leq \\ & \min \left\{ S_k^* - r_k + \min_{q \in \mathcal{L}, u \in N} \{I(r_k, r_q) + \max\{r_u - C_q^*, 0\} + p_u - R_q(r_u) | C_q^* \leq S_u^*, p_u < p_q\}, \right. \\ & \quad \left. S_k^* - r_k + \min_{q \in \mathcal{L}} \{I(r_k, r_q) + \alpha_q(r_q)\} - \epsilon, \alpha_i(C_k^*) - \epsilon, \min_{q \in \mathcal{L}} \{R_i(r_q) - p_q | S_k^* \leq S_q^* < C_i^*\} - \epsilon \right\}. \end{aligned}$$

Proof. The results follow from Lemmas 4.5 and 4.6. ■

Theorem 4.5 is possible because the temporal structure of scheduling problems determines the Δ value at which the schedule changes.

We now present a result that addresses issue I4 for Q3. This result illustrates the usefulness of finding bounds on parameter changes that maintain the job sequence. A substantial parameter change may be needed to alter the optimal sequence, while only a small parameter change may alter the optimal schedule. As a consequence, the ranges in the conditions of Theorem 4.5 are wider than similar ranges for the optimal schedule.

Theorem 4.6. *If either of the two conditions in the statement of Theorem 4.5 is satisfied, then a new optimal schedule can be found in $O(n)$ time.*

Proof. The optimality of the new schedule follows from Lemma 4.3 and Theorem 4.5. Given σ^* , the conditions in Theorem 4.5 can be verified in $O(n)$ time. ■

Observe that the SRPT algorithm of Miller and Schrage (1966) requires $O(n \log n)$ time to reconstruct an optimal schedule after the change in data. Thus, the use of the optimal job sequence permits a reduction in the time required to find a new optimal schedule.

4.3. Linear programming algorithms

Consider scheduling problems that are solved efficiently by some variant of linear programming. In general, the dual solution provides sensitivity analysis information. However, the linear programming formulation may obscure the answer to some sensitivity analysis questions. For instance, consider $1|r_j, \bar{d}_j, p_j = 1|\sum w_j C_j$. This scheduling problem can be efficiently formulated and solved as an assignment problem. The formulation uses decision variables x_{kt} , where $x_{kt} = 1$ if job k is processed at time t , and $x_{kt} = 0$ otherwise. The cost of completing job k at time t is $v_{kt} = w_k t$. If we change the weight of job k , then all of the objective function coefficients $v_{kr_k+1}, \dots, v_{k\bar{d}_k}$ change. This is an example of issue I2 where simultaneous changes occur due to the modelling process. Since these changes have a specific structure, we can improve on standard linear programming sensitivity analysis for this scheduling problem.

First, we address issue I2 by answering Q4.3 for simultaneous changes to several weights. We begin with two definitions (see Ahuja et al., 1993). Given a set of flows in a graph, the associated *residual network* is an identical graph, but with reduced arc capacities that represent the amount of additional flow that can be sent along each arc. The *node potentials* are the dual linear programming variables associated with the flow balance equations.

An efficient method for solving the assignment problem is the Cost Scaling Algorithm of Gabow and Tarjan (1989). Given integer data, this algorithm runs in $O(n^{2.5} \log(nc))$ time, where c is the maximum cost. For our scheduling problem, $c = \max_{1 \leq j \leq n} \{w_j \bar{d}_j\}$. We also assume integer data.

Theorem 4.7. *Let $w'_k = w_k + \Delta_k$, where Δ_k is integral, for $k = 1, \dots, n$, be parameter changes for $1|r_j, \bar{d}_j, p_j = 1|\sum w_j C_j$. Then, σ' can be found in $O(n^{2.5} \log(n \max_{1 \leq k \leq n} \{|\Delta_k| \bar{d}_k\}))$ time.*

Proof. Consider the residual network associated with an optimal solution for the original data set. Let λ_{ik} denote the reduced cost of arc (i, k) for the optimal node potential vector. Because the solution is optimal, $\lambda_{ik} \geq 0$ for every arc (i, k) in the residual network. The parameter changes imply that the absolute value of each cost in the assignment formulation changes by no more than $\max_{1 \leq k \leq n} \{|\Delta_k| \bar{d}_k\}$. Thus, after the parameter changes, $\lambda_{ik} \geq -\max_{1 \leq k \leq n} \{|\Delta_k| \bar{d}_k\}$ for all arcs (i, k) in the residual network. Therefore, we can apply additional iterations of the cost scaling algorithm, which finds a new optimal schedule in the stated time. ■

If the original optimal solution is not used, then resolving the scheduling problem by the Cost Scaling Algorithm requires $O(n^{2.5} \log(n \max_{1 \leq k \leq n} \{w_k + |\Delta_k| \bar{d}_k\}))$ time to answer Q4.3. This

time requirement may be substantially more than that in Theorem 4.7. Therefore, Theorem 4.7 is an example of issue I2.

We now consider a single parameter change of the form $w'_k = w_k + \Delta$, for some $k \in N$. This change increases the assignment costs from $v_{kt} = w_k t$ to $v'_{kt} = (w_k + \Delta)t$, for $t = r_k + 1, \dots, \bar{d}_k$. We formulate this scheduling problem as a matching problem on a bipartite graph. There are several closely related sensitivity analysis studies of the matching problem on a general graph. Weber (1981) and Derigs (1985) consider sensitivity analysis following the deletion of, or a weight change to, a single arc. A related problem is considered by Ball and Taverna (1985). Motivated by issue I2, we propose the following algorithm to answer Q4.3.

Algorithm WCCHG

0. Let σ^* be an optimal solution for $1|r_j, \bar{d}_j, p_j = 1|\sum w_j C_j$ formulated as an assignment problem. Solution σ^* assigns processing time C_j^* to job j , for $j = 1, \dots, n$. Let k denote the job that has a modified weight.
1. Change the assignment costs from v_{kt} to $v'_{kt} = (w_k + \Delta)t$, for $t = r_k + 1, \dots, \bar{d}_k$. Let $\hat{\sigma}$ be σ^* with job k and time C_k^* removed.
2. Starting with solution $\hat{\sigma}$, reoptimize the solution, where job k and time C_k^* are included, by using one augmenting path iteration of the Hungarian Algorithm (Kuhn, 1955).

Theorem 4.8. For some $k \in N$, let $w'_k = w_k + \Delta$ be a parameter change for $1|r_j, \bar{d}_j, p_j = 1|\sum w_j C_j$. Then, Algorithm WCCHG finds a new optimal solution in $O(n^2)$ time.

Proof. If $\hat{\sigma}$ is not optimal for the assignment subproblem defined by jobs $\{1, \dots, n\} \setminus \{k\}$ and time periods $\cup_{1 \leq j \neq k \leq n} \{r_j + 1, \dots, \bar{d}_j\} \setminus \{C_k^*\}$, then σ^* is not optimal for the original scheduling problem. In Step 2, we find an augmenting path of solution $\hat{\sigma}$ with minimal cost. The optimality of the solution follows from Papadimitriou and Steiglitz (1982).

Step 1 requires $O(n)$ time. Since job k is processed in one of the time periods $\{r_k + 1, \dots, \min\{r_k + n, \bar{d}_k\}\}$, Step 2 requires $O(n^2)$ time (Papadimitriou and Steiglitz, 1982). Thus, the overall time complexity of the sensitivity analysis in WCCHG is $O(n^2)$. ■

Answering Q4.3 with an efficient strongly polynomial time assignment algorithm (Ahuja et al., 1993) requires $O(n^3)$ time. Thus, Theorem 4.8 is an example of issue I2. Using the current optimal solution and Steps 1 and 2 of WCCHG, Q3 can be answered in less time than it takes to find the optimal schedule. Note that Karp and Orlin (1981) describe parametric shortest path algorithms for changes in cost that are similar to those described in Step 1 of WCCHG.

Corollary 4.1. For some $k \in N$, let $r'_k = r_k + \Delta$ or $\bar{d}'_k = \bar{d}_k + \Delta$ be a parameter change for $1|r_j, \bar{d}_j, p_j = 1|\sum w_j C_j$. Then, a procedure similar to Algorithm WCCHG finds a new optimal solution in $O(n^2)$ time.

We now consider a single parameter change of the form $w'_k = w_k + \Delta$ for some $k \in N$. This change may alter several of the $v_{kt} = w_k t$ costs (issue I2). Since $1|r_j, \bar{d}_j, p_j = 1|\sum w_j C_j$ can be formulated as a linear program, standard linear programming sensitivity analysis results are first considered. We provide conditions that are sufficient to maintain optimality.

Let S denote the set of times at which processing occurs in any feasible schedule that does not contain inserted idle time, where $|S| = n$. When there are n jobs and n times, a linear

programming assignment problem has $2n$ constraints. However, the constraint matrix does not have full row rank. A standard formulation eliminates one row, which makes the remaining rows linearly independent (e.g., see Gal, 1979). As a result, there are $n - 1$ basic variables with a value of zero. We construct an equivalent formulation in which all of the basic variables corresponding to jobs $1, \dots, n$ have a value of 1. We introduce a dummy job 0 and a dummy time -1 . The new problem has $(n + 1)^2$ variables and $2n + 2$ constraints. The cost of scheduling job 0 is 0 at time -1 and is large for all of the other n times. Thus, only job 0 is scheduled at time -1 . We augment S to include time -1 .

Delete the row $\sum_{t \in S} x_{0t} = 1$ to make the remaining rows linearly independent. Let A be the resulting matrix. Arrange the assignment constraints such that they appear in the order

$$\begin{aligned} \sum_{j=0}^n a_{jt} x_{jt} &= 1, \quad t \in S \\ \sum_{t \in S} a_{kt} x_{kt} &= 1, \quad k = 1, \dots, n. \end{aligned}$$

Let A_{jt} denote the column of the coefficient matrix corresponding to variable x_{jt} for $j \in \{0, \dots, n\}$ and $t \in S$. Each A_{jt} has at most two ones and the remaining entries are zero. Let x^* denote an optimal solution to $1|r_j, \bar{d}_j, p_j = 1| \sum w_j C_j$. Consider the basis B constructed from the columns A_{0t} , $t \in S$, and A_{kt} where $x_{kC_k^*}^* = 1$ and $k = 1, \dots, n$. Observe that this basis is optimal. The first $n + 1$ columns of B form an identity matrix above a matrix containing all zero entries. Each of the last n columns has exactly two entries of 1, one along the diagonal and one corresponding to the row associated with the time at which the job is completed. Let $(B^{-1}A_{kt})_k$ be the component of $B^{-1}A_{kt}$ corresponding to job k . Let $\lambda_{kt} \geq 0$ denote the reduced cost associated with nonbasic variable x_{kt}^* . By definition, this is the maximum decrease in the cost of the associated variable such that it remains nonbasic, assuming that no other parameter changes occur.

Suppose that we use either the Hungarian Algorithm or the Cost Scaling Algorithm to solve $1|r_j, \bar{d}_j, p_j = 1| \sum w_j C_j$. Both algorithms provide node potentials α_k and β_t for $t = r_k + 1, \dots, \bar{d}_k$, at optimality. Then, we can compute $\lambda_{kt} = v_{kt} - \alpha_k - \beta_t$ for a nonbasic variable in constant time for each $k \in N$ and $t \in S$. The next result addresses issue I2 by answering Q4.1.

Theorem 4.9. Consider the optimal solution and the reduced costs λ_{kt} for $1|r_j, \bar{d}_j, p_j = 1| \sum w_j C_j$ formulated as an assignment problem. For some $k \in N$, let $w_k' = w_k + \Delta$ be a parameter change for $1|r_j, \bar{d}_j, p_j = 1| \sum w_j C_j$. Schedule σ^ remains optimal if*

$$\underline{\Delta}_k = \max_{t=C_k^*+1, \dots, \bar{d}_k} \left\{ \lambda_{kt} / (C_k^* - t) \right\} \leq \Delta \leq \bar{\Delta}_k = \min_{t=r_k+1, \dots, C_k^*-1} \left\{ \lambda_{kt} / (C_k^* - t) \right\}.$$

Moreover, the values $\underline{\Delta}_k$ and $\bar{\Delta}_k$, for $k = 1, \dots, n$, can be found in $O(n^2)$ time.

Proof. We first consider $\bar{\Delta}_k$. From the discussion preceding this theorem, $x_{kt}^* = 0$ for $t \in \{r_k + 1, \dots, C_k^* - 1, C_k^* + 1, \dots, \bar{d}_k\}$. Also, since $v_{kt} = w_k t$, when w_k increases, scheduling job k at time t becomes more costly relative to other jobs. Thus, if σ^* is no longer optimal because of an increase in w_k , then x_{kt} becomes basic for some $t \in \{r_k + 1, \dots, C_k^* - 1\}$. We only need to find the value of Δ for which $\lambda_{kt} - (-\Delta t) - \Delta C_k^* (B^{-1}A_{kt})_k = 0$ (see, for example, Gal, 1979, or Hillier and Lieberman, 1995). By definition, when $\Delta \leq \bar{\Delta}_k$, the optimal solution does not

change. Hence, a sufficient condition for σ^* to remain optimal is

$$\Delta \leq \bar{\Delta}_k = \min_{t=r_k+1, \dots, C_k^*-1} \left\{ \lambda_{kt} / [C_k^* (B^{-1} A_{kt})_k - t] \right\}. \quad (2)$$

For a given k , consider the calculation of $\bar{\Delta}_k$ in (2). The matrix B^{-1} is the matrix B where each nonzero off diagonal entry in B is replaced by -1 . Also, each column of A_{kt} and each row of B^{-1} has at most two nonzero entries. The first nonzero entry of A_{kt} is associated with a coefficient in the equation $\sum_{j=0}^n a_{0t} x_{0t} = 1$ and the second nonzero entry is associated with a coefficient in the equation $\sum_{t \in S} a_{kt} x_{kt} = 1$. Since all entries below the main diagonal of B^{-1} are 0, it follows that $(B^{-1} A_{kt})_k = 1$. Substituting into (2) establishes the value of $\bar{\Delta}_k$.

Because there are $O(n)$ times t to consider, the computational work required to compute $\bar{\Delta}_k$ is $O(n)$. Thus, the total computational work requires $O(n^2)$ time because there are n jobs.

A similar analysis establishes the bound for $\bar{\Delta}_k$. ■

Since all parameter changes are in the same direction, the sensitivity range presented in Theorem 4.9 is wider than that given by the 100% rule (Bradley et al., 1977) and by linear programming analysis for a single cost change.

5. PLANNING FOR SENSITIVITY ANALYSIS

In many situations, sensitivity analysis is considered only after an optimal solution is found. However, the optimization approach taken and the computational effort required may change considerably if sensitivity analysis is anticipated (issue I6). Further, relevant sensitivity analysis questions and the appropriate procedure may depend on the parameters being modified. In this section, we first discuss situations where we should spend more time during the solution procedure to speed up subsequent sensitivity analysis. Then, we consider the selection of a different and more complicated solution procedure to facilitate subsequent sensitivity analysis. Next, we examine an example where the particular optimization procedure depends on the specific type of sensitivity analysis to be performed. Finally, we study particular solutions that are robust to certain types of sensitivity analysis.

First, as an example of issues I2 and I6, we discuss a situation where optimization is performed under less time pressure, or with less cost, than sensitivity analysis. Consider a production shop where the schedule for the next day can be run overnight. Time constraints during the night are less severe than those arising during the following day's production process. On the following day, we may need a quick evaluation of the effect of parameter changes on the optimal cost or optimal schedule. Consequently, we may be willing to use a less efficient algorithm overnight, if it allows us to answer sensitivity questions more quickly during the following day.

Theorem 5.1. For some $k \in N$, let $p'_k = p_k + \Delta_1$ and $w'_k = w_k + \Delta_2$ be parameter changes for $1 \leq j \leq n$. If π^* is no longer optimal, then π' and z'' can be found in $O(\log n)$ time.

Proof. Reindex the jobs such that $w_1/p_1 \geq \dots \geq w_n/p_n$. If π^* is no longer optimal, then from the WSPT rule, either $w'_k/p'_k > w_{k-1}/p_{k-1}$ or $w'_k/p'_k < w_{k+1}/p_{k+1}$. Assume that $w'_k/p'_k > w_k/p_k$. The other case is similar. Let $B_1 = \{j \mid w_j/p_j > w'_k/p'_k\}$, $B_2 = \{j \mid w'_k/p'_k \geq w_j/p_j \geq w_k/p_k\}$, and

$B_3 = \{j \mid w_k/p_k > w_j/p_j\}$. Using binary search over the integers in the interval $[0, n - |B_3|]$, we can find $|B_1|$ in $O(\log n)$ time. Placing job k in position $|B_1| + 1$ in the sequence, and moving job j one position later for all $j \in B_2$ creates a new optimal sequence (see Theorem 4.4). To represent this new sequence in constant time, we store σ^* in a doubly linked list and adjust only the links for job k and jobs adjacent to it. The resulting change in cost is

$$p_k \sum_{j \in B_2} w_j + (p'_k - p_k) \sum_{j \in B_3} w_j + w'_k \left(\sum_{j \in B_1} p_j + p'_k \right) - w_k \left(\sum_{j \in B_1} p_j + \sum_{j \in B_2} p_j + p_k \right).$$

The change in cost can be computed in constant time if $\sum_{j=i}^n w_j$ and $\sum_{j=1}^i p_j$ for $i = 1, \dots, n$, are precomputed during optimization. ■

Like Theorems 4.2 and 4.4, Theorem 5.1 is possible because the temporal structure of a schedule determines the cost change resulting from a pairwise interchange of jobs. It also motivates the search for other problems where the cost of a new optimal solution can be found without finding the new optimal solution itself.

To reduce the computational work of sensitivity analysis, the key idea is to precompute the partial sums $\sum_{j=i}^n w_j$ and $\sum_{j=1}^i p_j$ for $i = 1, \dots, n$, during optimization. Observe that the time complexity of the optimization procedure remains unchanged, since $\sum_{j=i}^n w_j$ and $\sum_{j=1}^i p_j$ for $i = 1, \dots, n$ can be computed in $O(n)$ time. However, the complexity of finding a new optimal sequence and its cost, to answer Q4.2 and Q4.3, is reduced from $O(n)$ to $O(\log n)$ time.

Motivated by issues I4 and I6, the question of when it may be worthwhile to use a different optimal procedure is examined for $1 \parallel \sum U_j$. The conventional way to solve this scheduling problem is by using the algorithm of Moore (1968). Because the jobs must be sorted in EDD order, the time complexity of this algorithm is $O(n \log n)$. To answer Q1, we propose a dynamic programming optimization procedure that requires $O(n^2)$ time. At the completion of optimization, we have the information necessary to perform sensitivity analysis.

Since the on-time jobs for $1 \parallel \sum U_j$ can be sequenced in EDD order in an optimal schedule, we reindex the jobs such that $d_1 \leq \dots \leq d_n$. Consider a network, G , consisting of nodes defined by the pair (i, j) , where exactly j jobs from the set $\{1, \dots, i\}$ are on time. In G , the only arcs from node $(i-1, j-1)$ are those to nodes $(i, j-1)$ and (i, j) , with distances 0 and p_i , respectively. The arc from $(i-1, j-1)$ to $(i, j-1)$ represents a decision that job i is late, and the arc from $(i-1, j-1)$ to (i, j) represents a decision that job i is on time. The shortest path from $(0, 0)$ to (i, j) is the minimum makespan required to schedule exactly j of jobs $1, \dots, i$ on time.

The following forward dynamic program computes this makespan. Let

$$f_i(j) = \text{minimum makespan required for } j \text{ jobs from the set } \{1, \dots, i\} \text{ to be on time}$$

$$= \begin{cases} \min\{f_{i-1}(j), f_{i-1}(j-1) + p_i\}, & \text{if } \min\{f_{i-1}(j), f_{i-1}(j-1) + p_i\} \leq d_i \\ \infty, & \text{otherwise.} \end{cases} \quad (3)$$

The boundary conditions are, for $i = 0, \dots, n$ and $j \leq i$,

$$f_i(j) = \begin{cases} 0, & \text{if } i = 0 \text{ and } j = 0 \\ \infty, & \text{otherwise.} \end{cases}$$

The optimal value of the recursion is $f^* = \max\{j \mid f_n(j) < \infty\}$, where $z^* = n - f^*$.

Next, we solve a backward recursion. The value function is

$$\begin{aligned} g_i(j) &= \text{latest time at which } j \text{ jobs from } \{1, \dots, i\} \text{ can complete on time,} \\ &\quad \text{such that at least } f^* \text{ jobs from } \{1, \dots, n\} \text{ can be on time} \\ &= \min\{d_i, \max\{g_{i+1}(j), g_{i+1}(j+1) - p_{i+1}\}\}. \end{aligned} \quad (4)$$

The boundary conditions are, for $i = 0, \dots, n$ and $j \leq i$,

$$g_i(j) = \begin{cases} d_i, & \text{if } i = n \text{ and } j = f^* \\ -\infty, & \text{otherwise.} \end{cases}$$

We now construct an algorithm to answer Q1 for a parameter change of the form $p'_k = p_k + \Delta$.

Algorithm DPSLACK

0. Reindex the jobs such that $d_1 \leq \dots \leq d_n$.
1. For $j \leq i$, solve (3) for $i = 1, \dots, n$, and (4) for $i = n-1, \dots, 0$.
2. For $i = 1, \dots, n$,
 If $\max_{j=0, \dots, i-1} \{g_i(j) - f_{i-1}(j)\} \geq 0$, then set $\bar{\Delta}_i = \infty$.
 Otherwise, set $\bar{\Delta}_i = \max_{j=0, \dots, i-1} \{g_i(j+1) - f_{i-1}(j) - p_i\}$.
3. For $i = 1, \dots, n$, set $\underline{\Delta}_i = \max\{0, \max_{j=0, \dots, i-1} \{g_i(j) - f_{i-1}(j)\}\} - p_i$.
4. Output $\bar{\Delta} = (\bar{\Delta}_1, \dots, \bar{\Delta}_n)$ and $\underline{\Delta} = (\underline{\Delta}_1, \dots, \underline{\Delta}_n)$.

Theorem 5.2. For some $k \in N$, let $p'_k = p_k + \Delta$ be a parameter change for $1 \parallel \sum U_j$. Then, Algorithm DPSLACK finds $\bar{\Delta}$ and $\underline{\Delta}$ for z^* .

Proof. We first consider $\bar{\Delta}$. The computation of $\max_{j=0, \dots, i-1} \{g_i(j) - f_{i-1}(j)\}$ in Step 2 determines whether there is a feasible path from node $(0, 0)$ to node (n, f^*) for which job k is not scheduled on time. That is, does there exist a j such that the arc from node $(k-1, j)$ to node (k, j) is on the feasible path? If so, then an increase of any size in p_k does not change the value of the optimal schedule since job k is not scheduled. Thus, $\bar{\Delta}_k = \infty$. Otherwise, every optimal schedule for the original data set schedules job k on time. In this case, the second computation in Step 2 calculates the maximum increase in p_k such that a feasible path still exists, that is, such that the optimal value remains unchanged. This maximum increase is $\bar{\Delta}_k$. Note that $g_i(j+1) - f_{i-1}(j) \geq p_i$ for any arc from node $(i-1, j)$ to node $(i, j+1)$ on a feasible path from node $(0, 0)$ to node (n, f^*) .

Next, we consider $\underline{\Delta}$. Observe that, given any feasible schedule with $f^* + 1$ on time jobs where p_k has been reduced by $|\underline{\Delta}|$, removing job k creates a feasible schedule with f^* on-time jobs. Hence, we find a schedule with f^* jobs on time and job k late, where $g_k(j) - f_{k-1}(j)$ is as large as possible. This provides the minimum time to process job k . Note that $g_i(j) - f_{i-1}(j) < p_i$ for any arc from node $(i-1, j)$ to node (i, j) on a feasible path from node $(0, 0)$ to node (n, f^*) ; otherwise, we can process job i on time. This creates a feasible solution with $f^* + 1$ on time jobs. When $g_k(j) - f_{k-1}(j) \leq 0$ for all j , every feasible schedule with f^* on-time jobs requires job k to be on time. Thus, there is no feasible schedule with $f^* + 1$ on-time jobs where $p'_k > 0$. ■

Note that the computations in Steps 2 and 3 are similar to the computations of the slack on an arc in the Critical Path Method for project management.

The use of DPSLACK instead of Moore's algorithm arises from the need to answer Q1. It requires $O(n^2)$ time to compute (3) and (4) in Step 1. An alternative procedure using Moore's algorithm works as follows. For each of the $O(n)$ jobs, a binary search procedure finds the smallest increase and the smallest decrease in the processing time that change z^* . At each value in the binary search procedure, Moore's algorithm requires $O(n \log n)$ time.

A final example of issue I6 concerns a situation where the choice of optimization approach is motivated by sensitivity analysis requirements. Consider $1||\sum w_j U_j$, which is binary *NP-hard* (Karp, 1972), but solvable in pseudopolynomial time by dynamic programming (Lawler and Moore, 1969). Two alternative dynamic programming algorithms are available. The first algorithm uses a state space defined by (job, schedule length) pairs, arc costs defined by job weights, and a value function that minimizes the total weight of late jobs. This algorithm requires $O(n \sum p_j)$ time. The second algorithm uses a state space defined by (job, total weight of late jobs) pairs, arc costs defined by job processing times, and a value function that minimizes the schedule length. This algorithm requires $O(n \sum w_j)$ time. If $\sum p_j$ and $\sum w_j$ are of approximately the same order of magnitude, then the running times of the two algorithms are similar. However, anticipating the need for a specific type of sensitivity analysis might justify the choice of one dynamic program over another. For example, using the first algorithm for optimization, an application of DPSLACK answers Q1 for a change in a weight, but not for a change in a processing time. This is because the arc costs are weights. A similar procedure that uses the second algorithm for optimization answers Q1 for a change in a processing time, but not for a change in a weight. This is because the arc costs are processing times. Therefore, the type of sensitivity analysis required might influence the type of optimization procedure used.

We now consider the issue of selecting among optimal schedules based on their robustness (issue I7). This is important in planning for a parameter change. There are a variety of scheduling problems where multiple optimal schedules occur frequently. The choice of an optimal schedule may greatly affect the robustness of sensitivity analysis results. An example is $P|pmtn|C_{\max}$. An optimal procedure schedules jobs in arbitrary index order on the lowest index machine with spare capacity (McNaughton, 1959). Since each ordering of the indices provides a different optimal schedule, there are many optimal schedules. We choose among the optimal schedules by finding one that is most robust with respect to π^* , that is, provides the widest range for Q1.

For all $k \in N$, the change in processing time, Δ_k , satisfies $-p_k \leq \Delta_k < \infty$. For a specified k , the following procedure finds an optimal schedule for $P|pmtn|C_{\max}$, where the job sequence remains optimal for any $\Delta_k \in [-p_k, \infty)$. Thus, the maximum optimality range is achieved.

Algorithm WIDE

0. Input $p_1, \dots, p_n$, job k , and the number of machines m .
1. Find $l = \operatorname{argmax}\{p_j | j \in N \setminus \{k\}\}$ and $C_{\max} = \max\{\sum_{j=1}^n p_j / m, p_l\}$.
Divide job k into m pieces, each with processing time p_k/m . Start processing the i th piece at time $s_i = (i-1)(C_{\max} - p_k/m)/(m-1)$ on machine M_i , for $i = 1, \dots, m$.
2. Let $R = \{j \in N \setminus \{k\} | p_j > C_{\max} - p_k/m\}$.
If $C_{\max} = \max \sum_{j=1}^n p_j / m$ or $C_{\max} = p_k$, then go to Step 4.
3. Process no job during the interval $[s_i + p_k/m, s_i + p_k/m + \min\{(C_{\max} - p_k)/(m-1), C_{\max} - (\sum_{j=1}^n p_j - p_l)/(m-1)\}]$ on machine M_i , for $i = 1, \dots, m-1$.

4. Schedule the jobs in R . Then, schedule the remaining jobs. Start processing each job in index order on the available machine with the lowest index as early as possible (as in McNaughton's Algorithm). If necessary, complete processing on the machine with the next higher index. Do not process a job when it is being processed on the previous machine.

While π^* remains unchanged, the following procedure adjusts σ^* after p_k changes by Δ_k .

Procedure PKADJ

1. If $\Delta_k < 0$, then reduce the processing time of each piece of job k by Δ_k/m . Also, decrease the start times of all jobs processed on the same machine after a given piece of k by $\min\{\Delta_k/m, t\}$, where a decrease of more than t results in simultaneous processing of a job on two machines.
2. If $\Delta_k > 0$ and $C_{\max} = \sum_{j=1}^n p_j/m$, then increase by an equal amount the processing times of all pieces of job k on all machines until a total of $\min\{\Delta_k, m(C_{\max} - p_k)/(m-1)\}$ is added. Also, increase by an equivalent amount the start times of all the jobs that are processed on the same machine after a given piece of k . Add any remaining processing time to the piece of job k on machine M_m .
3. If $\Delta_k > 0$ and $C_{\max} > \sum_{j=1}^n p_j/m$, then increase by equal amounts the processing times of all the pieces of job k on machines $M_1, \dots, M_{m-1}$. Continue until $v = \min\{\Delta_k, C_{\max} - p_k, (m-1)C_{\max} - \sum_{j=1}^n p_j + p_l\}$ total processing time is added.
 - a. When $v = C_{\max} - p_k$, add any remaining processing time to the piece of job k on M_m .
 - b. When $v = (m-1)C_{\max} - \sum_{j=1}^n p_j + p_l$, increase the processing time of each piece of job k by the same amount on all machines until a further total processing time of $\min\{\Delta_k - v, m(C_{\max} - p_k - v)/(m-1)\}$ is added. Also, increase by an equivalent amount the start times of all jobs that are processed on the same machine after a given piece of k . Add any remaining processing time to the piece of job k on M_m .

In Step 2 of PKADJ, when the amount of processing added to job k is $m(C_{\max} - p_k)/(m-1)$, then $p'_k = C_{\max}$. Any additional processing of a piece of k on one of the machines $M_1, \dots, M_{m-1}$ results in simultaneous processing of k on two machines. Consequently, the remaining processing time is added to the piece of job k on machine M_m . While the makespan increases, the sequence remains optimal because $p'_k = C_{\max}$.

In Step 3 of PKADJ, when $\Delta_k > 0$ and $C_{\max} > \sum_{j=1}^n p_j/m$, the first stage inserts additional processing time into the idle time intervals created in Step 3 of WIDE on machines $M_1, \dots, M_{m-1}$. If $v = C_{\max} - p_k$, then the total processing time of jobs $N \setminus \{l\}$ is small enough that p'_k becomes equal to $C_{\max}$ at least as soon as any of the first $m-1$ machines processes jobs for $C_{\max}$ units of time. Any remaining processing time of $\Delta_k - (C_{\max} - p_k)$ is added to the piece of k on machine M_m . If $v = (m-1)C_{\max} - \sum_{j=1}^n p_j + p_l$, then idle time of size $C_{\max} - (\sum_{j=1}^n p_j - p_l)/(m-1)$ is inserted on each of the first $m-1$ machines in Step 3 of WIDE. After the first part of Step 3 of PKADJ, all this idle time is filled by processing pieces of k . Also, all machines complete at $C_{\max}$. Then, an equal amount of processing time is added to the piece of k on each of the m machines until the total processing time of k on the first $m-1$ machines is $C_{\max} - p_k/m$. Since the piece of k that is processed on machine M_m starts processing at time $C_{\max} - p_k/m$, job k is processed throughout the entire processing time interval. Any remaining

processing time of $\Delta_k - m(C_{\max} - p_k - v) / (m - 1)$ is added to the piece of job k on machine M_m .

Theorem 5.3. For some $k \in N$, let $p'_k = p_k + \Delta_k$ be a parameter change for $P|pmtn|C_{\max}$. Then, in $O(m + n)$ time, Algorithm WIDE finds an optimal schedule for which π^* remains optimal for $\Delta_k \in [-p_k, \infty)$.

Proof. WIDE assigns each job $j \in N \setminus \{k\}$ to at most two machines, where those machines have consecutive indices.

We first consider the case when $\Delta_k < 0$. If we reduce each piece of job k in PKADJ, then the job pieces that are processed after a piece of k on any given machine all start earlier by the same amount. Also, on any given machine, the job pieces that are processed before a piece of k do not change processing times. Suppose job j is processed on machines M_i and M_{i+1} . Since the jobs in R are assigned first in WIDE, no processing of job j occurs on M_{i+1} after the completion of the piece of k on M_i . Further, from the definition of R and Step 4 of WIDE, each job $j \in R$ starts on M_i no later than the start time of the piece of k on M_{i+1} . Hence, the time between when j finishes processing on M_{i+1} and a piece of j starts after the piece of k on M_i is at least $C_{\max} - p_j$. On each machine, the completion time becomes smaller by an equal amount until either $p'_k = 0$ or the makespan is equal to the processing time of some job. The scheduling procedure in Step 1 of PKADJ ensures that no pair of jobs is processed simultaneously. Consequently, the sequence remains feasible. Because either each machine has the same amount of processing or the makespan is equal to the processing time of some job, the sequence remains optimal.

For the case when $\Delta_k > 0$ and $C_{\max} = \sum_{j=1}^n p_j / m$, the job pieces that are processed after a piece of k on any given machine all start later by the same amount. Also, on any given machine, the job pieces that are processed before a piece of job k do not change processing times. Therefore, the starting time of some job $j \neq k$ increases only if it starts after the piece of k on a given machine M_i . If job j is also processed on machine M_{i+1} , it is processed before the piece of k on that machine. Consequently, simultaneous processing of two pieces is only possible for job k . Step 2 of PKADJ prevents this from happening until a total processing time of $m(C_{\max} - p_k) / (m - 1)$ is added. Also from Step 2, any remaining processing of job k occurs on machine M_m , when no other job is being processed. Since no job is processed simultaneously on multiple machines, the sequence remains feasible. Because the completion time of each machine increases by the same amount until $p'_k = C_{\max}$, the sequence remains optimal.

When $\Delta_k > 0$ and $C_{\max} > \sum_{j=1}^n p_j / m$, the first set of processing time increases does not increase the completion time of any machine. The analysis is now similar to the previous case.

Steps 0, 2, and 4 of WIDE each require $O(n)$ time. Step 1 requires $O(m + n)$ time. Step 3 requires $O(m)$ time. Thus, the overall time complexity of WIDE is $O(m + n)$. ■

The schedule found by WIDE is not the only schedule that satisfies the conditions of Theorem 5.3. Further refinements exist where jobs other than k can have wider sensitivity ranges. Theorem 5.3 is possible because the temporal structure of scheduling problems determines the Δ value at which the processing of a job on parallel machines becomes simultaneous and therefore infeasible. It motivates the development of methods for choosing between multiple optimal solutions, based on their robustness, in other optimization problems.

We now establish the companion result that there is no optimal sequence where the processing time of any *arbitrary* job can be made infinitely large and the sequence remains optimal. This negative result is also an example of issue I7.

Theorem 5.4. For some $k \in N$, let $p'_k = p_k + \Delta_k$ be a parameter change for $P|pmtn|C_{\max}$. If $n > m$, then there is no sequence that remains optimal for each k and all $\Delta_k > 0$.

Proof. When Δ_k becomes sufficiently large, $C_{\max} = p'_k$. Once this happens, the schedule remains optimal only if job k is processed throughout the time interval $[0, C_{\max}]$. This implies that job k must start processing at time 0. Because there are only m machines, only m jobs can start at time 0. Consequently, when $n > m$, there is some job k that does not start at time 0. Thus, the sequence becomes nonoptimal for Δ_k sufficiently large. ■

It is possible to circumvent our argument in Theorem 5.4 by including job pieces with zero processing times. However, this is not realistic since scheduling, transporting, and setting up a zero processing time job piece in a production shop is costly, and confusing to the workforce. Theorem 5.4 motivates the investigation in other optimization problems of whether results of the type in Theorem 5.3 are possible.

6. INTRACTABLE PROBLEMS

In this section, we examine sensitivity analysis for intractable scheduling problems. In Section 6.1, motivated by issue I8, we explore the relationship between the computational complexity of a scheduling problem and the computational complexity of associated sensitivity analysis questions. In Section 6.2, motivated by issue I9, we suggest approaches to performing heuristic sensitivity analysis for intractable scheduling problems.

6.1. Computational complexity

In general, the computational complexity of performing sensitivity analysis for scheduling problems is not resolved. This is because knowledge of σ^* may affect the work required. The only general result is given by Van Hoesel and Wagelmans (1999) who show that, under reasonable conditions, finding a sensitivity analysis range (question Q1) for an intractable zero-one optimization problem is itself intractable. This implies that answering Q1 for changes in *weights* is intractable, when the scheduling problem is intractable. The next two results show that it may be difficult to develop efficient sensitivity analysis techniques for any intractable scheduling problem. Let $\Psi = \{r_j, p_j, w_j, d_j, J_j, prec\}$ be the set of problem class parameters. The following result addresses issue I8.

Theorem 6.1. Suppose that problem class P is polynomially solvable. Also, suppose that when problem data of type $\omega \in \Psi$ is added to P , the resulting problem class $\hat{P}$ is NP-hard.

- i. Given only the optimal cost, answering Q2 for problem class $\hat{P}$ is NP-hard.
- ii. Given the optimal schedule and its cost, there does not exist a polynomial time algorithm that answers Q3 for problem class $\hat{P}$ unless $P = NP$.

Proof. For part i, consider an arbitrary instance, I , of problem class $\hat{P}$. Remove all data of type ω . Solve this instance of P in polynomial time. Next, change one type ω parameter of one job to match a job in I , and apply a hypothetical polynomial time algorithm to answer Q2.

Repeat this procedure for the other jobs. Consequently, the optimal cost for I can be found in polynomial time. This contradicts our assumption about the complexity of $\hat{P}$ and establishes that answering Q2 is binary NP -hard.

The proof is similar for part ii, except that following each parameter change we find in polynomial time both a new optimal schedule and its cost. ■

Theorem 6.1 motivates the investigation, especially in areas of optimization that contain many closely related problems, of whether similar results can be obtained.

Corollary 6.1. Suppose that problem class P is pseudopolynomially solvable. Also, suppose that when problem data of type $\omega \in \Psi$ is added to P , the resulting problem class $\hat{P}$ is unary NP -hard.

- i. Given only the optimal cost, answering Q2 for problem class $\hat{P}$ is unary NP -hard.*
- ii. Given the optimal schedule and its cost, there does not exist a pseudopolynomial time algorithm that answers Q3 for problem class $\hat{P}$ unless $P = NP$.*

Proof. Similar to Theorem 6.1 except that polynomial is replaced by pseudopolynomial. ■

The results in Theorem 6.1 and Corollary 6.1 address issue I8 for Q2 and Q3.

Remark 6.1 Performing sensitivity analysis on the processing times for most NP -hard classes of scheduling problems is NP -hard.

When the problem data added is p_j , the instance of P being considered is the one where all processing times are zero. Thus, each job is processed at the earliest possible time. Assuming that a feasible solution exists, it generally can be found in polynomial time. When the constraints create an infeasibility, as in the case when $r_i > \bar{d}_i$ for some $i \in N$ or when the precedence constraints generate a cyclic graph, this also can usually be checked in polynomial time. Exceptions to the remark include scheduling problems where the processing times are specified by the problem definition, such as the unary NP -hard $1|prec, p_j = 1|\sum w_j C_j$ (Lawler, 1978).

6.2. Sensitivity analysis using heuristic methods

The results in Section 6.1 motivate the use of heuristic sensitivity analysis methods that can estimate the effect of parameter changes in polynomial time. We consider the issue of how far from the optimal cost a previously optimal schedule can be as a result of a parameter change. This information may make it unnecessary to resolve the problem with the new data if a sufficiently small deviation from optimality is guaranteed.

Two examples are discussed. The first example considers $P2||\sum w_j C_j$. This scheduling problem is known to be binary NP -hard (Bruno et al., 1974). The optimization version is solved in pseudopolynomial time by Lawler and Moore (1969). We find bounds for a single parameter change. The value of the bounds and the computational effort depend on the position of the job in σ^* .

Reindex the jobs such that $w_1/p_1 \geq \dots \geq w_n/p_n$. The following result provides an example of issues I5 and I9 applied to Q2. As discussed in Section 1, Condition b in the theorem statement prevents the overtaking of job completion times.

Theorem 6.2. For some $k \in N$, let $p'_k = p_k + \Delta$ be a parameter change for $P2 \parallel \sum w_j C_j$. For σ^* , let B be the set of jobs that are processed after job k and are on the same machine as job k . Let Δ be such that

- a. $w_{k-1}/p_{k-1} \geq w_k/p'_k \geq w_{k+1}/p_{k+1}$.
- b. For job i processed on machine M_1 and job j processed on machine M_2 , $C'_i(\sigma^*) \leq C'_j(\sigma^*)$ if and only if $C_i^* \leq C_j^*$, for $1 \leq i \neq j \leq n$.

Then,

- i. If $\Delta > 0$, then

$$\left(z^* + \Delta w_k + \Delta \sum_{i \in B} w_i \right) / \left(z^* + \Delta \sum_{i=k}^n w_i \right) \leq z'(\sigma^*) / z'' \leq \left(z^* + \Delta w_k + \Delta \sum_{i \in B} w_i \right) / \left(z^* + \Delta w_k \right).$$

- ii. If $\Delta < 0$, then

$$\left(z^* + \Delta w_k + \Delta \sum_{i \in B} w_j \right) / \left(z^* + \Delta w_k \right) \leq z'(\sigma^*) / z'' \leq \left(z^* + \Delta w_k + \Delta \sum_{i \in B} w_i \right) / \left(z^* + \Delta \sum_{i=k}^n w_i \right).$$

Proof. By construction. ■

Theorem 6.2 is possible because the temporal structure of scheduling problems provides in Condition b the exact Δ value at which the completion time of one job overtakes that of another. It is also an example of approximate sensitivity analysis, which has potential application to any optimization problem.

Observe that $|B|$ is smaller when job k is processed later in the schedule. Consequently, Theorem 6.2 is a position dependent result in two distinct ways. First, the computational effort required depends on the location of the job in the schedule. The later job k is processed, the less computational work is needed. Second, the value of the bound depends on the location of job k in the schedule. The later job k is processed, the smaller the bound is on $z'(\sigma)/z''$ if $\Delta > 0$ and the greater the bound is if $\Delta < 0$. In the limit, if k is the last job on its machine, then $B = \emptyset$. Thus, $z^* = z''$ (see also Theorem 3.1) when $\Delta > 0$. Also, if all jobs $i > k$ are in B , then $w_k + \sum_{i \in B} w_i = \sum_{i=k}^n w_i$. Thus, $z^* = z''$ when $\Delta < 0$.

We continue our discussion of issue I9 by considering an example of heuristic sensitivity analysis for $P2 \parallel C_{\max}$. This scheduling problem is binary *NP*-hard (Karp, 1972). We develop a procedure that finds the interval over which a parameter can change, such that the ratio of the new cost of σ^* to the new optimal cost is bounded above by a specific value. An instance is presented where the bound for Q1 is attainable. We show how to modify σ^* to obtain a heuristic schedule with the same error bound for a larger parameter change. Also, we describe a class of problem instances for which the heuristic procedure finds an exact upper bound for Q1.

We examine the case where the processing time of some job increases. The analysis is similar if the processing time decreases. First, we present the LF (Largest Fit) heuristic that finds a less than exact packing of a subset of items with sizes $a_1, \dots, a_n$ into a knapsack of size b .

Subroutine LF($a_1, \dots, a_n; b$)

0. Let $N = \{1, \dots, n\}$. Reindex the items such that $a_1 \geq \dots \geq a_n$.
1. Find $j = \min\{i \in N \mid a_i < b\}$.
If j does not exist, then set LF = b , which is the unused space, and terminate.
2. Set $b = b - a_j$ and $N = N \setminus \{j\}$. Go to Step 1.

If $\sum_{i=1}^n a_i \geq b$, then this subroutine finds a solution with a value that is at least $1/2$ of the optimal value. Consider an example where $b = 2$ and where there are three jobs that have sizes $1 + \epsilon$, 1 and 1 . For this example, the bound of $1/2$ is asymptotically attainable.

Without loss of generality, assume that job n is processed on M_2 and has an increase in processing time of $\Delta > 0$. Let c_i denote the completion time of M_i , for $i = 1, 2$. (The schedule is implied by the context.) Let $\bar{\Delta}$ denote the maximum increase in p_n such that π^* is still optimal. When $\Delta > \bar{\Delta}$, there is a swap of a subset of the jobs on M_1 with a subset of those on M_2 , that gives a schedule with smaller makespan. Our heuristic considers some of the possible swaps. If the heuristic finds an improvement, then this establishes that π^* is no longer optimal.

Heuristic SWAP

0. Reindex the machines such that job n is processed on machine M_2 in σ^* . Let A denote the set of jobs that σ^* assigns to machine M_1 , and p_A the vector of processing times of these jobs. Let $B = N \setminus (A \cup \{n\})$.
1. Set $\Delta_1 = \text{LF}(p_A; p_i)$, where $i = \arg\min_{j \in B} \{p_j\}$.
Set $\Delta_2 = \text{LF}(p_A; \min_{j \in B \setminus \{i\}} \{p_j\})$.
Set $\Delta_3 = \text{LF}(p_A; c_2 - p_n)$.
2. $\Delta^u = \min\{\Delta_1, \Delta_2, \Delta_3\} + c_1 - c_2$.

In Step 1, SWAP considers three different packings of the jobs of A into subsets of B . If $\Delta > \Delta^u$, then π^* is no longer optimal. We can swap the selected jobs of A with the corresponding jobs on machine M_2 to reduce the makespan.

Step 0 of SWAP requires $O(n)$ time. The time requirement for Step 1 of SWAP is the time requirement for LF. Step 2 of SWAP requires constant time. Indexing the jobs in Step 0 of LF requires $O(n \log n)$ time. In Step 1 of LF, the selection from an ordered list requires constant time. Since there are at most n selections, the time requirement for LF is $O(n \log n)$. As a result, the overall time complexity of SWAP is $O(n \log n)$.

To establish the worst-case performance of SWAP, several preliminary results are needed. The first result provides values of Δ for which σ^* remains optimal, answering Q1.

Theorem 6.3. Let $p'_n = p_n + \Delta$ be a parameter change for $P2 \parallel C_{\max}$, where job n is processed on machine M_2 . If $0 \leq \Delta \leq c_1 - c_2$, then $z'(\sigma^*) = z''$.

Proof. By definition of σ' , $z'' = z'(\sigma') \leq z'(\sigma^*)$. If $0 \leq \Delta \leq c_1 - c_2$, then $z'(\sigma^*) = z(\sigma^*)$. Thus, $z'(\sigma^*) = z(\sigma^*) \leq z(\sigma') = z'(\sigma')$. Therefore, $z'(\sigma^*) = z'(\sigma')$. ■

Theorem 6.3 implies that if $c_1 > c_2$, then p_n can be increased so that the two machines complete at the same time and the optimal solution does not change. It results from the continuous variability in the temporal structure of scheduling problems.

The next result answers Q1 by finding a case when SWAP provides an exact bound for $\bar{\Delta}$.

Theorem 6.4. *SWAP finds $\Delta^u = \bar{\Delta}$ for parameter change $p'_n = p_n + \Delta$ when $n \leq 4$.*

Proof. Suppose that M_2 processes only job n . Since $c_2 = p_n$, π^* remains optimal for any $\Delta \geq 0$.

If M_2 processes two jobs, then M_1 processes at most two jobs. Therefore, Δ_1 is the size of the maximal packing of $c_2 - p_n$. Since the only possible swap is to interchange the job in B with those in A , SWAP must find the swap associated with $\bar{\Delta}$.

If M_2 processes three jobs, then M_1 processes only one job. Therefore, Δ_1 , Δ_2 , and Δ_3 are the sizes of the maximal packing of each job in B and of $c_2 - p_n$. Since these are the only possible swaps, SWAP must find the one associated with $\bar{\Delta}$. ■

To answer Q2, we analyze the maximum relative cost error for σ^* that can result from a parameter change. Let $\Delta^s = \min\{\Delta_1, \Delta_2, \Delta_3\}$.

Theorem 6.5. *For $P2\|C_{\max}$, SWAP provides an upper bound for $\bar{\Delta}$ for parameter change $p'_n = p_n + \Delta$ in $O(n \log n)$ time. If $\Delta \leq \Delta^u$, then $z'(\sigma^*)/z'' \leq 8/7$. Further, this bound is attainable.*

Proof. Scale the jobs of each problem instance so that SWAP determines that $\Delta^s = 2$. Although fractional processing times may result, this does not affect our analysis. From Theorem 6.4, we assume that $n \geq 5$.

Suppose that M_2 processes only job n in σ^* . Job n remains the only job processed by M_2 in an optimal schedule as Δ increases. Thus, π^* remains optimal.

Now, suppose that M_2 processes two jobs in σ^* . SWAP fails to find $\bar{\Delta}$ only when the maximal packing of $c_2 - p_n$ is not found. This can happen only when SWAP selects a larger job from A that is not in the optimal packing. Then, SWAP is unable to select smaller jobs that are part of the optimal packing. There must be at least two smaller jobs in the optimal packing that are not selected by SWAP, because the optimal packing has a total size larger than the one found by SWAP. Since $\Delta^s = \min\{\Delta_1, \Delta_2, \Delta_3\} = 2$ by assumption, each of these smaller jobs has size at least two; otherwise, one of these jobs would be in the packing found by SWAP. Because A contains at least three jobs, each of which has size at least two, $c_1 \geq 6$. Since $\Delta^s = 2$, $z'(\sigma^*) \leq c_1 + 2$. Because the total processing time in the modified scheduling problem is at least $2c_1 + 2$, we have that $z'' \geq (2c_1 + 2)/2 = c_1 + 1$. Consequently, $z'(\sigma^*)/z'' \leq (c_1 + 2)/(c_1 + 1) \leq 8/7$.

When M_2 processes three jobs, SWAP finds a packing for B and for each job in B . Thus, SWAP considers the subset of B that is part of an optimal swap. Since SWAP always finds an optimal packing when $|A| \leq 2$ and the packing found by SWAP is suboptimal, it must be the case that $|A| \geq 3$. This implies that $c_1 \geq 6$. Then, using the analysis for the case when M_2 processes two jobs, we conclude that $z'(\sigma^*)/z'' \leq 8/7$.

Finally, suppose that M_2 processes at least four jobs. Since $\Delta^s = 2$, each job in B must have size at least two. Because $|B| \geq 3$, $\sum_{j \in B} p_j \geq 6$. When $c_1 < \sum_{j \in B} p_j$, the makespan can be reduced by moving job n from M_2 to M_1 . This implies that σ^* is not optimal. Therefore, $c_1 \geq \sum_{j \in B} p_j \geq 6$. Similar to the case when M_2 processes two jobs, we conclude that $z'(\sigma^*)/z'' \leq 8/7$.

To show that the bound of $8/7$ is attainable, consider an instance with six jobs, where the processing times are 3, 3, 2, 2, 2, and 1 for jobs 1 through 6, respectively. In σ^* , jobs 1 and 2 are processed on machine M_1 , and jobs 3, 4, 5, and 6 are processed on machine M_2 . SWAP gives $\Delta^u = \Delta_1 + c_1 - c_2 = 2 + 6 - 7 = 1$. As a result, $z'(\sigma^*) = 8$. However, $\Delta^u = 1$ implies that $p'_6 = p_6 + \Delta^u = 2$. As a result, σ' has jobs 1, 3, and 4 on machine M_1 , and jobs 2, 5, and 6 on machine M_2 . Since $z'' = 7$, we have that $z'(\sigma^*)/z'' = 8/7$.

The computational complexity of finding $\bar{\Delta}$ follows from our discussion of SWAP. ■

Theorem 6.5 is an example of approximate sensitivity analysis (issue I9), with potential application to any optimization problem. It results from the continuous variability in the temporal structure of scheduling problems.

To answer Q2 and Q3, SWAP provides a new heuristic schedule that bounds the optimal cost. Let σ^s be the schedule generated when the swap associated with Δ^s is performed on σ^* .

Theorem 6.6. Let $p'_n = p_n + \Delta$ be a parameter change for $P2\|C_{\max}$. If $\Delta^u \leq \Delta \leq \Delta^u + 7\Delta^s/3$, then $z'(\sigma^s)/z'' \leq 8/7$. Further, within the interval $[\Delta^u, \Delta^u + 7\Delta^s/3]$, this bound is attainable only for $\Delta \in \{\Delta^u, \Delta^u + 7\Delta^s/3\}$.

Proof. When $\Delta = \Delta^u$, then $z'(\sigma^s) = z'(\sigma^*)$ by construction. The swap moves a total processing time of Δ^s from M_2 to M_1 . Thus, $c_1(\sigma^s) = c_2(\sigma^s) + \Delta^s$ when $\Delta = \Delta^u$. Let z^{LP} denote the optimal cost of the linear programming relaxation for a given Δ . As Δ increases in the interval $[\Delta^u, \Delta^u + \Delta^s]$, $z'(\sigma^s) - z^{LP}$ decreases. When $\Delta = \Delta^u + \Delta^s$, $z'(\sigma^s) - z^{LP} = 0$.

From the proof of Theorem 6.5, when $\Delta = \Delta^u$, $c_2'(\sigma^*)/c_1'(\sigma^*) = [c_1(\sigma^*) + \Delta^s]/c_1(\sigma^*) \leq 8/6$. This implies that $\Delta^s \leq c_1(\sigma^*)/3$. Therefore, if $\Delta^u \leq \Delta \leq \Delta^u + 7\Delta^s/3$, then

$$\begin{aligned} z'(\sigma^s)/c_1(\sigma^s) &\leq [c_1(\sigma^*) + 7\Delta^s/3]/[c_1(\sigma^*) + \Delta^s] \\ &\leq [c_1(\sigma^*) + 7c_1(\sigma^*)/9]/[c_1(\sigma^*) + c_1(\sigma^*)/3] = \frac{8}{6}. \end{aligned}$$

Thus, $z'(\sigma^s)/z'' \leq z'(\sigma^s)/z^{LP} \leq z'(\sigma^s)/[(c_1(\sigma^s) + c_2(\sigma^s))/2] \leq 8/7$.

Since $z'(\sigma^s) - z^{LP}$ is strictly decreasing in the interval $[\Delta^u, \Delta^u + \Delta^s]$ and strictly increasing in the interval $[\Delta^u + \Delta^s, \Delta^u + 7\Delta^s/3]$, if $z'(\sigma^s)/z'' = 8/7$, then $\Delta \in \{\Delta^u, \Delta^u + 7\Delta^s/3\}$.

We now show that the bound of 8/7 is attainable for $\Delta \in \{\Delta^u, \Delta^u + 7\Delta^s/3\}$. The example from the proof of Theorem 6.5 shows that the bound is attainable for $\Delta = \Delta^u$. Consider an instance with six jobs, where the processing times are 10, 8, 6, 6, 6, and 1 for jobs 1 through 6, respectively. In σ^* , jobs 1 and 2 are processed on M_1 , and jobs 3, 4, 5, and 6 are processed on M_2 . SWAP gives $\Delta^u = 5$ and $\Delta^s = 6$. For $\Delta = \Delta^u + 7\Delta^s/3 = 19$, we have that $p'_6 = 20$. Schedule σ^s processes jobs 1, 2, and 3 on machine M_1 and jobs 4, 5, and 6 on M_2 . Hence, $z'(\sigma^s) = 32$. However, σ' processes jobs 1, 3, 4, and 5 on M_1 and jobs 2 and 6 on M_2 , and $z'' = 28$. Thus, for $\Delta = \Delta^u + 7\Delta^s/3$, we have that $z'(\sigma^s)/z'' = 8/7$. ■

Theorem 6.6 results from the continuous variability in the temporal structure of scheduling problems. Note that the $O(n)$ time heuristic of He et al. (2000) for $P2\|C_{\max}$ provides a bound of 12/11. However, this heuristic only holds for a given data set. By contrast, the results in Theorems 6.5 and 6.6 apply for all values of Δ within the specified ranges. It would be interesting to extend our discussion of issue I9 by finding problems in other areas of optimization where similar bounds can be obtained for an entire range of data.

7. CONCLUDING REMARKS

This paper represents a first attempt at a systematic study of sensitivity analysis for scheduling problems. We examine several examples where it is possible to compute a new optimal schedule

efficiently by using the existing optimal schedule. Situations are identified where sensitivity analysis results for an optimal schedule can be obtained efficiently by using an optimal sequence. We also present several situations where it is worthwhile to perform additional computations or a different type of computations, to facilitate later sensitivity analysis. In some cases, the results depend on the position of the job with changed parameters in the original optimal sequence. We also discuss the computational complexity of answering sensitivity analysis questions for intractable scheduling problems. For an intractable scheduling problem, we establish worst-case performance bounds on the relative increase above optimal cost when using an original optimal schedule after a parameter change. Finally, we extend the previous analysis to derive heuristic solutions, also with guaranteed worst-case performance, for a parameter change within a larger range.

Our analysis can be applied to numerous situations that are not discussed due to lack of space. We now mention some examples. The methodology of Theorems 4.3 and 4.4 can be applied to $1||L_{\max}$, for which Jackson (1955) shows that an earliest due date sequence is optimal. An analysis similar to that in Theorem 4.2 can be applied to $P2||\sum C_j$. The analysis of release date changes for $1|r_j, pmtn||\sum C_j$ in Theorems 4.5 and 4.6 can also be applied to changes in processing times for that scheduling problem. The idea of pre-planning for sensitivity analysis discussed in Section 5 can also be used for intractable scheduling problems. As a final example, the heuristic sensitivity analysis for $P2||C_{\max}$ in Section 6.2 can be applied to similar parallel machine scheduling problems.

Our paper provides suggestions for sensitivity analysis that can be performed for a variety of scheduling problems, and identifies many interesting open issues. We hope that our research will encourage the development of further sensitivity analysis issues and approaches for scheduling and other optimization problems.

ACKNOWLEDGMENTS

This work is supported in part by National Science Foundation grants DMI-9821033 and DMI-9808437, and in part by the Summer Fellowship Program of the Fisher College of Business, The Ohio State University.

REFERENCES

- Ahuja, R. K., T. L. Magnanti, and J. B. Orlin, *Network Flows: Theory, Algorithms and Applications*. Prentice Hall, Englewood Cliffs, NJ, 1993.
- Anderson, D. R., D. J. Sweeney, and T. A. Williams, *Contemporary Management Science With Spreadsheets*. South-Western College Publishing, Cincinnati, OH, 1999.
- Ball, M. O. and R. Taverna, "Sensitivity analysis for the matching problem and its use in solving matching problems with a single side constraint," *Ann. Oper. Res.*, **4**, 25–56 (1985).
- Bazaraa, M. S., J. J. Jarvis, and H. D. Sherali, *Linear Programming and Network Flows*, 2nd edition. John Wiley, New York, NY, 1990.
- Bradley, S. P., A. C. Hax, and T. L. Magnanti, *Applied Mathematical Programming*. Addison-Wesley, Reading, MA, 1977.
- Bruno, J. L., E. G. Coffman, Jr, and R. Sethi, "Scheduling independent tasks to reduce mean finishing time," *Commun. ACM*, **17**, 382–387 (1974).
- Camm, J. D. and T. H. Burwell, "Sensitivity analysis in linear programming models with common inputs," *Decision Sci.*, **22**, 512–518 (1991).
- Chakravarti, N. and A. P. M. Wagelmans, "Calculation of Stability Radii for Combinatorial Optimization Problems," *Oper. Res. Lett.*, **23**, 1–7 (1998).
- Conway, R. W., W. L. Maxwell, and L. W. Miller, *Theory of Scheduling*. Addison-Wesley, Reading, MA, 1967.
- Cook, W., A. M. H. Gerards, A. Schrijver, and E. Tardos, "Sensitivity theorems in integer linear programming," *Math. Program.*, **34**, 251–264 (1986).

- Cozzens, M. and F. S. Roberts, "Greedy algorithms for T -colorings of graphs and the meaningfulness of conclusions about them," *J. Combin. Inform. Systems Sci.*, **16**, 286–299 (1991).
- Daniels, R. L. and P. Kouvelis, "Robust scheduling to hedge against processing time uncertainty in single-stage production," *Manage. Sci.*, **41**, 363–376 (1995).
- Dawande, M. W. and J. N. Hooker, "Inference-based sensitivity analysis for mixed integer programming," *Oper. Res.*, **48**, 623–634 (2000).
- Derigs, U., "Postoptimal analysis for matching problems," *Method Oper. Res.*, **49**, 215–221 (1985).
- Derigs, U., *Programming in Networks and Graphs*. Springer, New York, 1988.
- Federgruen, A. and M. Tzur, "Minimal forecast horizons and a new planning procedure for the general dynamic lot sizing model: nervousness revisited," *Oper. Res.*, **42**, 456–468 (1994).
- Gabow, H. N. and R. E. Tarjan, "Faster scaling algorithms for network problems," *SIAM J Comput.*, **18**, 1013–1036 (1989).
- Gal, T., *Postoptimal Analyses, Parametric Programming and Related Topics*. McGraw-Hill, Great Britain, 1979.
- Garey, M. R. and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. Freeman, San Francisco, 1979.
- Geoffrion, A. M. and R. Nauss, "Parametric and postoptimality analysis in integer linear programming," *Manage. Sci.*, **23**, 453–466 (1977).
- Graham, R. L., "Bounds on the performance of scheduling algorithms," in E. G. Coffman, Jr. (ed.), *Computer and Job-Shop Scheduling Theory*, John Wiley, New York, 1975.
- Graham, R. L., E. L. Lawler, J. K. Lenstra, and A. H. G. Rinnooy Kan, "Optimization and approximation in deterministic machine scheduling: A survey," *Ann. Discrete Math.*, **5**, 287–326 (1979).
- Greenberg, H. J., "An annotated bibliography for post-solution analysis in mixed integer programming and combinatorial optimization," in D. L. Woodruff (ed.), Kluwer Academic Publishers, Dordrecht, The Netherlands, pp. 97–148, 1998.
- Gusfield, D., *Sensitivity Analysis for Combinatorial Optimization*. Ph.D. Thesis. UCN/ERL M80/22, Research Memorandum, Electronics Research Laboratory, Berkeley, CA, (1980).
- Hall, N. G., "Single and multiple processor models for minimizing completion time variance," *Nav. Res. Logist. Q.*, **33**, 49–54 (1986).
- He, Y., H. Kellerer and V. Kotov, "Linear compound algorithms for the partitioning problem," *Nav. Res. Logist.*, **47**, 593–601 (2000).
- Hillier, F. S. and G. J. Lieberman, *Introduction to Operations Research*. 6th edition. McGraw-Hill, New York, 1995.
- Jackson, J. R., *Scheduling a Production Line to Minimize Maximum Tardiness*. Research Report 43, Management Science Research Project, University of California, Los Angeles, 1955.
- James, R. J. W. and J. T. Buchanan, "Robustness of single machine scheduling problems to earliness and tardiness penalty errors," *Ann. Oper. Res.*, **76**, 219–232 (1998).
- Kanet, J. J., "Minimizing the average deviation of job completion times about a common due date," *Nav. Res. Logist. Q.*, **28**, 643–651 (1981).
- Karp, R. M., "Reducibility among combinatorial problems," in R. E. Miller and J. W. Thatcher (eds), *Complexity of Computer Computations*, Plenum Press, New York, pp. 85–103, 1972.
- Karp, R. M. and J. B. Orlin, "Parametric shortest path algorithms with an application to cyclic staffing," *Discrete Appl. Math.*, **3**, 37–45 (1981).
- Klein, D. and S. Holm, "Integer programming post-optimal analysis with cutting planes," *Manage. Sci.*, **25**, 64–72 (1979).
- Kolen, A. W. J., A. H. G. Rinnooy Kan, C. P. M. van Hoesel, and A. P. M. Wagelmans, Sensitivity analysis of list scheduling heuristics *Discrete Appl. Math.*, **55**, 145–162 (1994).
- Kouvelis, P., R. L. Daniels, and G. Vairaktarakis, "Robust Scheduling of a Two-Machine Flow Shop with Uncertain Processing Times," *IIE Transactions*, **32**, 421–432 (2000).
- Kouvelis, P., and G. Yu, *Robust Discrete Optimization and its Application*. Kluwer Academic Publishers, Dordrecht, 1996.
- Kravchenko, S. A., Y. N. Sotskov, and F. Werner, "Optimal schedules with infinitely large stability radius," *Optimization*, **33**, 271–280 (1995).
- Kuhn, H. W., "The Hungarian method for the assignment problem," *Nav. Res. Logist. Q.*, **2**, 83–97 (1955).
- Lawler, E. L., "Sequencing jobs to minimize total weighted completion time subject to precedence constraints," *Ann. Discrete Math.*, **2**, 75–90 (1978).
- Lawler, E. L. and J. M. Moore, "A functional equation and its application to resource allocation and sequencing problems," *Manage. Sci.*, **16**, 77–84 (1969).
- Mahadev, N. V. R., A. Pekeč, and F. S. Roberts, "On the meaningfulness of optimal solutions to scheduling problems: can an optimal solution be nonoptimal?," *Oper. Res.*, **46**, S120–S134 (1998).
- Marsten, R. E. and T. L. Morin, "Parametric integer programming: The right hand side case," in P. Hammer et al. (eds), *Annals of Discrete Mathematics: Studies in Integer Programming*, Elsevier, North Holland, 1977.
- McNaughton, R., "Scheduling with deadlines and loss functions," *Manage. Sci.*, **6**, 1–12 (1959).

- Moore, J. M., "An n job, one machine sequencing algorithm for minimizing the number of late jobs," *Manage. Sci.*, **15**, 102–109 (1968).
- Papadimitriou, C. H. and K. Steiglitz, *Combinatorial Optimization: Algorithms and Complexity*. Prentice-Hall, Englewood Cliffs, NJ, 1982.
- Piper, C. J. and A. A. Zoltners, "Some easy postoptimality analysis for zero-one programming," *Manage. Sci.*, **22**, 759–765 (1976).
- Rinnooy Kan, A. H. G., *Machine Scheduling Problems: Classification, Complexity and Computations*. Nijhoff, The Hague, The Netherlands, 1976.
- Roberts, F. S., "Meaningfulness of conclusions from combinatorial optimization," *Discrete Appl. Math.*, **29**, 221–241 (1990).
- Roodman, G. M., "Postoptimality analysis in zero-one programming by implicit enumeration," *Nav. Res. Logist. Q.*, **19**, 435–447 (1972).
- Schrage, L. E., "A proof of the optimality of the shortest remaining processing time discipline," *Oper. Res.*, **16**, 687–690 (1968).
- Schrage, L. E. and L. A. Wolsey, "Sensitivity analysis for branch and bound integer programming," *Oper. Res.*, **33**, 1008–1023 (1985).
- Seeger, A., "Subgradients of optimal-value functions in dynamic programming: The case of convex systems without optimal paths," *Mathematics Oper. Res.*, **21**, 555–575 (1996).
- Smith, W. E., "Various optimizers for single stage production," *Nav. Res. Logist. Q.*, **3**, 59–66 (1956).
- Sotskov, Y. N., "Stability of an optimal schedule," *Eur. J. Oper. Res.*, **55**, 91–102 (1991).
- Sotskov, Y. N., V. K. Leontev, and E. N. Gordeev, "Some concepts of stability analysis in combinatorial optimization," *Discrete Appl. Math.*, **58**, 169–190 (1995).
- Sotskov, Y. N., A. P. M. Wagelmans, and F. Werner, "On the calculation of the stability radius of an optimal or an approximate schedule," *Ann. Oper. Res.*, **83**, 213–252 (1998).
- Steele, D. C., "The nervous MRP system: How to do battle," *Prod. Invent. Manage.*, **16**, 83–89 (1973).
- Tovey, C. A., "Rescheduling to minimize makespan on a changing number of identical processors," *Nav. Res. Logist. Q.*, **33**, 717–724 (1986).
- Tovey, C. A., "A simplified anomaly and reduction for precedence constrained multiprocessor scheduling," *SIAM J. Discrete Math.*, **3**, 582–584 (1990).
- Van Hoesel, C. P. M., and A. Wagelmans, "On the complexity of postoptimality analysis of 0/1 programs," *Discrete Appl. Math.*, **91**, 251–263 (1999).
- Wagelmans, A. P. M., *Sensitivity Analysis in Combinatorial Optimization*. PhD Thesis, Erasmus University, Rotterdam, Netherlands, 1990.
- Wagner, H. M., "Global sensitivity analysis," *Oper. Res.*, **43**, 948–969 (1995).
- Weber, G. M., "Sensitivity analysis of optimal matchings," *Networks*, **11**, 41–56 (1981).
- Wendell, R. E., "Tolerance approach to sensitivity analysis in linear programming," *Manage. Sci.*, **31**, 564–578 (1985).
- Wendell, R. E., "Sensitivity analysis revisited and extended," *Decision Sci.*, **23**, 1127–1142 (1992).
- Wondolowski, F. R. Jr., "A generalization of Wendell's tolerance approach to sensitivity analysis in linear programming," *Decision Sci.*, **22**, 792–810 (1991).
- Wolsey, L. A., "Integer programming duality: Price functions and sensitivity analysis," *Math. Program.*, **20**, 173–195 (1981).